

Past-Future Scheduler for LLM Serving under SLA Guarantees

Ruihao Gong^{*}
Beihang University
Beijing, China
gongruihao@buaa.edu.cn

Shihao Bai^{*}
SenseTime
Shanghai, China
baishihao@sensetime.com

Siyu Wu^{*}
Beihang University
Beijing, China
wusiyu@buaa.edu.cn

Yunqian Fan
SenseTime
Shanghai, China
fanyunqian@sensetime.com

Zaijun Wang
SenseTime
Shanghai, China
wangzaijun@sensetime.com

Xiuhong Li
Peking University
Beijing, China
lixihong@pku.edu.cn

Hailong Yang[†]
Beihang University
Beijing, China
hailong.yang@buaa.edu.cn

Xianglong Liu[†]
Beihang University
Beijing, China
xlliu@buaa.edu.cn

Abstract

The exploration and application of Large Language Models (LLMs) is thriving. To reduce deployment costs, continuous batching has become an essential feature in current service frameworks. The effectiveness of continuous batching relies on an accurate estimate of the memory requirements of requests. However, due to the diversity in request output lengths, existing frameworks tend to adopt aggressive or conservative schedulers, which often result in significant overestimation or underestimation of memory consumption. Consequently, they suffer from harmful request evictions or prolonged queuing times, failing to achieve satisfactory throughput under strict Service Level Agreement (SLA) guarantees (a.k.a. goodput), across various LLM application scenarios with differing input-output length distributions. To address this issue, we propose a novel Past-Future scheduler that precisely estimates the peak memory resources required by the running batch via considering the historical distribution of request output lengths and calculating memory occupancy at each future time point. It adapts to applications with all types of input-output length distributions, balancing the trade-off between request queuing and harmful evictions, thereby consistently achieving better goodput. Furthermore, to validate the effectiveness of the proposed scheduler, we developed a high-performance LLM serving framework, LightLLM, that implements the Past-Future scheduler. Compared to existing aggressive or conservative schedulers, LightLLM demonstrates superior goodput, achieving up to 2-3 $\times$ higher goodput than other schedulers under heavy loads. LightLLM is open source to boost the research in such direction (<https://github.com/ModelTC/lightllm>).

CCS Concepts: • Computing methodologies $\rightarrow$ Machine learning; Parallel computing methodologies.

Keywords: LLM Inference, Scheduling Optimization

1 Introduction

The burgeoning field of Large Language Models (LLMs) marks a watershed in technology, vastly enhancing potential applications across diverse domains such as chatbots[10, 21], summarization [18, 24, 26], code generation [9], and advanced reasoning [35, 36]. Models such as GPT-4 [22], Claude [1], and Llama [32] exemplify this trend, harnessing hundreds of billions of parameters for exceptional performance. However, the above breakthroughs come with high deployment expenses, especially under Service Level Agreement (SLA) guarantees. SLA guarantees, which constrain the first response time of requests and the maximum response interval between outputs, are crucial to user experience. For real-world LLM services, throughput under SLA guarantees, also known as goodput, is the well-adopted metric that accurately reflects the system’s throughput capacity. Take ChatGPT for example, with 100 million weekly active users, it strives to maintain stable and scalable services, even with a robust infrastructure of 10,000 GPUs. In such a context, optimizing system goodput with limited hardware resources is not only advantageous but also essential, which helps to reduce cost and expand the reach of LLMs.

To reduce costs and enhance performance, various innovative approaches have been developed, each targeting different aspects of the system. The primary focus of these efforts is to improve the inference backend and optimize the serving schedule. For improving the inference backend, existing works such as FlexGEN [27] demonstrate the power of parallel processing. Using pipeline parallelism, FlexGEN significantly increases inference throughput on lower-end

^{*}Equal contribution.

[†]Corresponding author.

SLA 保证下 LLM 服务的过去未来调度程序

Ruihao Gong*
Beihang University
Beijing, China
gongruihao@buaa.edu.cn

Yunqian Fan
SenseTime
Shanghai, China
fanyunqian@sensetime.com

Hailong Yang[†]
Beihang University
Beijing, China
hailong.yang@buaa.edu.cn

Shihao Bai*
SenseTime
Shanghai, China
baishihao@sensetime.com

Zaijun Wang
SenseTime
Shanghai, China
wangzaijun@sensetime.com

Siyu Wu*
Beihang University
Beijing, China
wusiyu@buaa.edu.cn

Xiuhong Li
Peking University
Beijing, China
lixihong@pku.edu.cn

Xianglong Liu[†]
Beihang University
Beijing, China
xlliu@buaa.edu.cn

抽象的

大型语言模型 (LLM) 的探索和应用正在蓬勃发展。为了降低部署成本，连续批处理已成为当前服务框架的基本功能。连续批处理的有效性依赖于对请求内存需求的准确估计。然而，由于请求输出长度的多样性，现有框架倾向于采用激进或保守的调度器，这通常会导致显着高估或低估内存消耗。因此，在具有不同输入输出长度分布的各种 LLM 应用场景中，它们会遭受有害的请求驱逐或延长的排队时间，无法在严格的服务级别协议 (SLA) 保证（又名良好吞吐量）下实现令人满意的吞吐量。为了解决这个问题，我们提出了一种新颖的过去未来调度器，它通过考虑请求输出长度的历史分布并计算每个未来时间点的内存占用来精确估计运行批处理所需的峰值内存资源。它适应具有所有类型的输入输出长度分布的应用程序，平衡请求排队和有害驱逐之间的权衡，从而持续实现更好的吞吐量。此外，为了验证所提出的调度程序的有效性，我们开发了一个高性能的 LLM 服务框架 LightLLM，它实现了过去-未来调度程序。与现有的激进或保守调度程序相比，LightLLM 表现出卓越的吞吐量，在重负载下比其他调度程序高出 2-3x 的吞吐量。LightLLM 是开源的，旨在促进这一方向的研究 (<https://github.com/ModelTC/lightllm>)。

CCS 概念: • 计算方法 → 机器学习; 并行计算方法。

关键词: LLM推理、调度优化

1 简介

大型语言模型 (LLM) 的新兴领域标志着技术的分水岭，极大地增强了聊天机器人 [10, 21]、摘要 [18, 24, 26]、代码生成 [9] 和高级推理 [35, 36] 等不同领域的潜在应用。GPT-4 [22]、Claude [1] 和 Llama [32] 等模型体现了这一趋势，利用数万亿个参数来实现卓越的性能。然而，上述突破伴随着高昂的部署费用，特别是在服务级别协议 (SLA) 保证下。SLA 保证限制了请求的首次响应时间和输出之间的最大响应间隔，对于用户体验至关重要。对于现实世界的 LLM 服务，SLA 保证下的吞吐量（也称为有效吞吐量）是广泛采用的指标，可以准确反映系统的吞吐量。以 ChatGPT 为例，每周有 1 亿活跃用户，即使拥有 10,000 个 GPU 的强大基础设施，它也努力保持稳定和可扩展的服务。在这样的背景下，利用有限的硬件资源优化系统吞吐量不仅是有利的，而且也是必要的，这有助于降低成本并扩大大语言模型覆盖范围。

为了降低成本并提高性能，人们开发了各种创新方法，每种方法都针对系统的不同方面。这些工作的主要重点是改进推理后端并优化服务计划。为了改进推理后端，FlexGEN [27]等现有工作展示了并行处理的强大功能。利用管道并行性，FlexGEN 显着提高了低端的推理吞吐量。

*Equal contribution.

[†]Corresponding author.

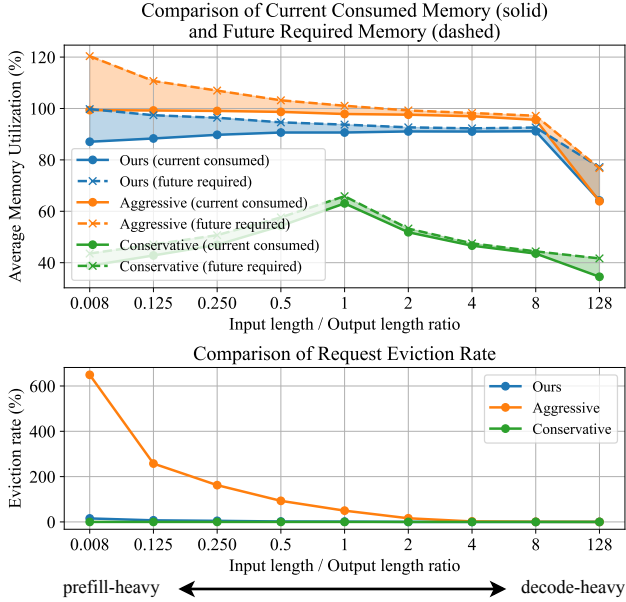


Figure 1. Comparison of current consumed memory (solid), future required memory (dashed), and request eviction rate ($>100\%$ means one request is evicted multiple times on average) for different schedulers under various input-output length distributions. Lower memory utilization indicates lower efficiency, while future required memory exceeds the memory capacity will lead to request evictions afterward.

processing units. Other works such as Flash Attention [6], Flash Decoding, and their advanced iteration, Flash Attention V2 [5] and Flash Decoding++ [11], have been devoted in creating high-performance kernels. These works are particularly focused on optimizing the Attention module, a core component of LLMs. In addition, GPTQ [8] and OS+ [37] utilize quantization to accelerate the inference. Furthermore, PageAttention [13] is a notable development for memory management in LLM, designed to prevent memory fragmentation during request serving. For optimizing the serving schedule, the scheduling strategies are crucial for system throughput. Traditional fixed batch size scheduling is inadequate for LLMs, which cannot handle varying input lengths and generate unpredictable output sizes, leading to unnecessary response delay. Continuous batching [40] has become a key component of current frameworks to address the above limitation by adjusting the batching granularity at each request to each decoding step, thereby reducing the memory and computation waste. The key to the success of continuous batching lies in accurately estimating the memory requirements of the requests. To estimate the request memory requirement, the existing request schedulers can be classified into two categories: 1) the conservative scheduler that overestimates memory consumption, and 2) the aggressive scheduler that underestimates memory consumption.

Despite the performance achieved by existing schedulers, they still struggle to achieve satisfactory goodput across diverse LLM service scenarios. For example, both conservative and aggressive schedulers can easily breach SLA constraints under high concurrency and thus lead to low goodput with poor user experience. As shown in Figure 1, the conservative schedulers [12, 17] often queue requests based on the assumption that each request will generate up to a pre-set maximum number of new tokens which rarely occurs in practice. In addition, they overlook that the requests in the running batch will dynamically finish at different future time points, and thus lead to considerably low memory utilization and long first response latency. On the other hand, the aggressive schedulers [13] attempt to incorporate as many incoming requests as possible into the decoding process. Although it can always achieve high memory utilization, such an over-optimistic approach can lead to the peak memory required in the future by the running batch exceeding the hardware memory capacity, resulting in more frequent request evictions. In such cases, it results in response interruptions, particularly when deploying decode-heavy LLM services that require longer outputs, such as the popular ChatGPT o1-preview with reasoning capabilities.

To achieve optimal goodput for LLM services across varying request length distributions, we propose a novel Past-Future Scheduler. This scheduler takes into account both the historical output length distribution and the future required memory demand of the running batch, allowing it to schedule queued requests at optimal time points. It also strikes a balance between queue time and the frequency of request evictions, ultimately enhancing the performance of the serving system.

Specifically, the proposed scheduler consists of two components: predicting output length distribution and estimating future required memory. The output length distribution prediction component involves dynamically estimating the most likely final output length of requests within the running batch, based on the distribution of historical request output lengths and their current output lengths. It relies on the observation that actual output lengths of the requests are often less than the preset maximums, and the output distribution of historical requests remains stable within adjacent time windows. Whereas, the future-required memory estimation component involves dynamically calculating the memory usage required by the running batch at each future step after updating the output length prediction for each request. It considers the memory released by completed requests at future time points, thereby determining the future required memory needed for the entire current running batch. Therefore, the proposed scheduler can precisely assess the impact of adding a new request to the running batch on future required memory at different time steps. By balancing queuing time and the frequency of request eviction, it ensures requests are scheduled at the optimal time points. As shown in

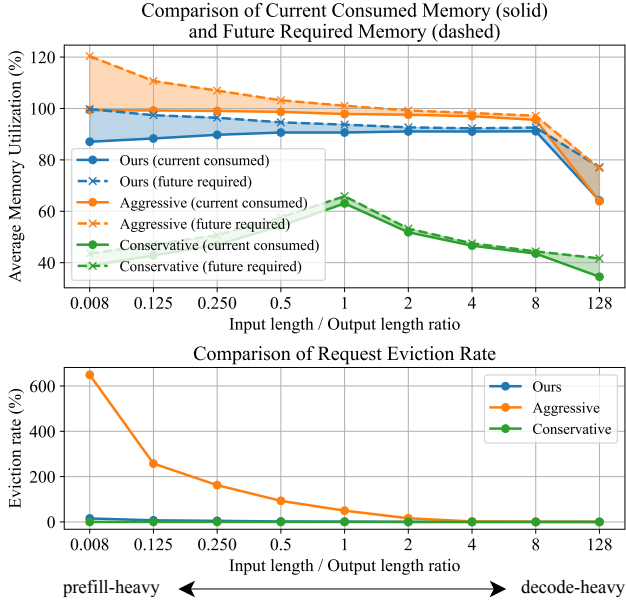


图 1. 不同调度器在各种输入输出长度分布下当前消耗的内存（实线）、未来所需的内存（虚线）和请求驱逐率（>100% 表示一个请求平均被驱逐多次）的比较。内存利用率较低表明效率较低，而未来所需的内存超过内存容量将导致随后的请求驱逐。

处理单位。其他工作，例如 Flash Attention [6]、Flash Decoding 及其高级迭代 Flash Attention V2 [5] 和 Flash Decoding++ [11]，一直致力于创建高性能内核。这些工作特别侧重于优化注意力模块，这是大语言模型组件。此外，GPTQ [8] 和 OS+ [37] 利用量化来加速推理。此外，PageAttention [13] 是 LLM 中内存管理的一个值得注意的发展，旨在防止请求服务期间的内存碎片。为了优化服务调度，调度策略对于系统吞吐量至关重要。传统的固定批量大小调度对于 LLM 来说是不够的，它无法处理变化的输入长度并生成不可预测的输出大小，从而导致不必要的响应延迟。连续批处理[40]已成为当前框架的关键组成部分，通过调整每个解码步骤的每个请求的批处理粒度来解决上述限制，从而减少内存和计算浪费。连续批处理成功的关键在于准确估计请求的内存需求。为了估计请求内存需求，现有的请求调度程序可以分为两类：1) 高估内存消耗的保守调度程序，2) 低估内存消耗的激进调度程序。

尽管现有调度程序取得了一定的性能，但它们仍然难以在不同的 LLM 服务场景中实现令人满意的吞吐量。例如，保守和激进的调度程序都可以在高并发下轻松突破 SLA 约束，从而导致低吞吐量和较差的用户体验。如图 1 所示，保守的调度程序 [12, 17] 通常基于每个请求将生成预设最大数量的新令牌的假设来对请求进行排队，这在实践中很少发生。此外，他们忽略了正在运行的批处理中的请求将在未来的不同时间点动态完成，从而导致相当低的内存利用率和较长的首次响应延迟。另一方面，积极的调度程序[13]尝试将尽可能多的传入请求合并到解码过程中。虽然它总能实现较高的内存利用率，但这种过于乐观的方法可能会导致运行批次超出硬件内存容量而导致未来所需的峰值内存，从而导致更频繁的请求驱逐。在这种情况下，它会导致响应中断，特别是在部署需要较长输出的大量解码 LLM 服务时，例如具有推理功能的流行 ChatGPT o1-preview。

为了在不同的请求长度分布上实现 LLM 服务的最佳吞吐量，我们提出了一种新颖的过去未来调度程序。该调度程序考虑了历史输出长度分布和正在运行的批处理的未来所需的内存需求，使其能够在最佳时间点调度排队的请求。它还在队列时间和请求驱逐频率之间取得平衡，最终提高服务系统的性能。

具体来说，所提出的调度程序由两个组件组成：预测输出长度分布和估计未来所需的内存。输出长度分布预测组件涉及根据历史请求输出长度及其当前输出长度的分布动态估计正在运行的批次内请求的最可能的最终输出长度。它依赖于观察到请求的实际输出长度通常小于预设的最大值，并且历史请求的输出分布在相邻时间窗口内保持稳定。而未来所需的内存估计组件涉及在更新每个请求的输出长度预测后动态计算正在运行的批处理在每个未来步骤所需的内存使用量。它考虑未来时间点已完成的请求释放的内存，从而确定整个当前运行批次所需的未来所需内存。因此，所提出的调度程序可以精确评估在不同时间步骤向正在运行的批处理添加新请求对未来所需内存的影响。通过平衡排队时间和请求驱逐的频率，它确保请求被安排在最佳时间点。如图所示

Figure 1, our method maintains high hardware memory utilization, with future required memory always approaching hardware capacity, while preventing evictions.

To demonstrate the effectiveness of the proposed Past-Future scheduler, we also designed a new LLM serving framework, LightLLM. With the help of the Past-Future scheduler, our LightLLM has been deployed to support services on a cluster with thousands of GPUs, providing various real-world LLM services such as code generation, human-like dialogue, long document analysis, and multimodal conversation. It can serve tens of millions of user requests and process billions of tokens per day under strict SLA constraints.

Specifically, this paper makes the following contributions:

- We discover the similarity of the LLM requests output length distribution within adjacent time windows, and propose a method to predict the output length distribution based on recent historical requests.
- We propose an accurate estimation method for the future required memory of LLM requests, and combine it with our output length prediction method to propose the Past-Future Request Scheduler.
- We develop a high-performance LLM service framework, LightLLM, that implements the Past-Future scheduler in a real production system for serving tens of millions of user requests under strict SLA constraints.
- We evaluate LightLLM with comprehensive experiments to demonstrate our Past-Future scheduler can achieve better goodput performance on multiple hardware platforms.

2 Background and Challenges

2.1 LLM Inference and KV Cache

Upon receiving a request consisting of a list of input prompt tokens, LLMs generate the output text in two stages: "prefill" and "decode". In the "prefill" stage, the model processes the input prompt tokens in parallel, producing an output token. Subsequently, the "decode" stage follows an autoregressive generation paradigm and generates output tokens one by one. Specifically, the input prompt tokens and the newly generated tokens are concatenated together and fed into the LLM to produce a new output token. This process is repeated iteratively until the end token is encountered or the maximum output length (*max_new_tokens*) is reached. However, inference with all tokens as input in each generation will lead to a lot of unnecessary computations, since in LLM the subsequent generated tokens will not affect the activation value of the previous token due to the presence of the attention mask. To speed up it, the Key and Value tensors of the input prompt tokens and the previously generated tokens are cached in GPU memory, known as KV cache. This caching mechanism allows the model to process only the latest input token when generating a new token, significantly enhancing token generation speed. With the KV cache, there is no

need to recalculate the input prompt tokens and previously generated tokens during each decoding process. Only the newly generated tokens need to be fed into the LLM for the next step, reducing computational overhead and improving inference latency. However, while the KV cache can enhance inference speed, it also leads to significant GPU memory usage, making the system efficiency become memory-bound and limiting throughput performance. Thus, managing the KV cache has become a key challenge in deploying high-performance LLM services.

2.2 Memory Management

Current LLM services, such as FasterTransformer [19] and ORCA [40], typically store the KV cache in contiguous GPU memory space. When processing a request, they frequently allocate GPU memory based on the maximum potential output length, as the exact output length is hard to predict, leading to severe memory fragmentation and hampering the overall throughput. To address this, vLLM [13] introduced PagedAttention, an attention algorithm that used block memory management, inspired by the classical virtual memory and paging techniques in operating systems. PageAttention reduces memory fragmentation by storing the KV cache in non-contiguous memory blocks. Despite the notable advances brought by memory management, current LLM frameworks underestimate the impact of the request scheduler on GPU memory utilization and system performance, and as a result, still fail to achieve optimal goodput under SLA constraints.

2.3 Continuous Batching

Batching multiple requests for inference is an effective way to improve the performance of LLM service. However, it presents greater challenges compared to traditional deep learning workloads, as it involves iteratively generating the final output text. If multiple requests with varying input lengths are statically grouped into a fixed batch size, padding them to match the length of the longest request leads to additional memory and computational waste. Moreover, due to the unpredictable output length of each request, earlier completed requests may be delayed while waiting for the unfinished ones. The continuous arrival of new requests may be queued and cannot be processed in a timely manner, resulting in a significant waste of both GPU memory and computational resources.

To address this issue, Continuous Batching, also known as Dynamic Batching or iteration-level scheduling, has been widely adopted in current LLM service frameworks. With customized attention operators and combined with memory management, when processing a batch of requests, there is no need to pad inputs to the maximum length or wait for all requests to finish. Instead, in each iteration, completed requests are filtered out promptly, and new requests are dynamically added to the batch based on the available GPU

图 1, 我们的方法保持了较高的硬件内存利用率, 未来所需的内存始终接近硬件容量, 同时防止驱逐。

为了证明所提出的过去-未来调度程序的有效性, 我们还设计了一个新的 LLM 服务框架 LightLLM。在 Past-Future 调度器的帮助下, 我们的 LightLLM 已经部署在具有数千个 GPU 的集群上支持服务, 提供各种现实世界的 LLM 服务, 例如代码生成、类人对话、长文档分析和多模态对话。它可以在严格的 SLA 约束下服务数千万个用户请求并每天处理数十亿个代币。

具体来说, 本文做出以下贡献:

- 我们发现相邻时间窗口内 LLM 请求输出长度分布的相似性, 并提出了一种根据最近的历史请求预测输出长度分布的方法。
- 我们提出了一种准确估计 LLM 请求未来所需内存的方法, 并将其与我们的输出长度预测方法相结合, 提出了过去未来请求调度器。
- 我们开发了一个高性能的 LLM 服务框架 LightLLM, 它在实际的生产系统中实现了过去未来调度程序, 以便在严格的 SLA 约束下服务数千万个用户请求。
- 我们通过综合实验评估 LightLLM, 以证明我们的过去未来调度程序可以在多个硬件平台上实现更好的吞吐量性能。

2 背景与挑战

2.1 LLM 推理和 KV 缓存

在收到包含输入提示标记列表的请求后, LLM 分两个阶段生成输出文本: “预填充”和“解码”。在“预填充”阶段, 模型并行处理输入提示标记, 生成输出标记。随后, “解码”阶段遵循自回归生成范例并一一生成输出令牌。具体来说, 输入提示标记和新生成的标记被连接在一起并输入 LLM 以生成新的输出标记。迭代重复此过程, 直到遇到结束标记或达到最大输出长度 (max_new_tokens)。然而, 在每一代中将所有令牌作为输入进行推理将导致大量不必要的计算, 因为在 LLM 中, 由于注意力掩码的存在, 后续生成的令牌不会影响前一个令牌的激活值。为了加快速度, 输入提示标记的 Key 和 Value 张量以及之前生成的标记被缓存在 GPU 内存中, 称为 KV 缓存。这种缓存机制允许模型在生成新令牌时仅处理最新的输入令牌, 从而显著提高令牌生成速度。有了 KV 缓存, 就没有了

每次解码过程中都需要重新计算输入提示 token 和之前生成的 token。只需将新生成的令牌输入到 LLM 中即可进行下一步, 从而减少计算开销并改善推理延迟。然而, 虽然 KV 缓存可以提高推理速度, 但它也会导致显着的 GPU 内存使用, 使系统效率受到内存限制并限制吞吐量性能。因此, 管理 KV 缓存已成为部署高性能 LLM 服务的关键挑战。

2.2 内存管理

当前的 LLM 服务, 例如 FasterTransformer [19] 和 ORCA [40], 通常将 KV 缓存存储在连续的 GPU 内存空间中。在处理请求时, 它们经常根据最大潜在输出长度来分配 GPU 内存, 因为确切的输出长度很难预测, 从而导致严重的内存碎片并阻碍整体吞吐量。为了解决这个问题, vLLM [13] 引入了 PagedAttention, 这是一种使用块内存管理的注意力算法, 受到操作系统中经典虚拟内存和分页技术的启发。PageAttention 通过将 KV 缓存存储在不连续的内存块中来减少内存碎片。尽管内存管理带来了显着的进步, 但当前的 LLM 框架低估了请求调度程序对 GPU 内存利用率和系统性能的影响, 因此仍然无法在 SLA 约束下实现最佳吞吐量。

2.3 连续配料

批处理多个推理请求是提高 LLM 服务性能的有效方法。然而, 与传统的深度学习工作负载相比, 它提出了更大的挑战, 因为它涉及迭代生成最终输出文本。如果将具有不同输入长度的多个请求静态分组为固定的批量大小, 则填充它们以匹配最长请求的长度会导致额外的内存和计算浪费。此外, 由于每个请求的输出长度不可预测, 较早完成的请求可能会在等待未完成的请求时被延迟。不断到来的新请求可能会被排队而无法及时处理, 从而导致 GPU 内存和计算资源的大量浪费。

为了解决这个问题, 连续批处理, 也称为动态批处理或迭代级调度, 在当前的 LLM 服务框架中被广泛采用。通过定制的注意力算子并结合内存管理, 在处理一批请求时, 无需将输入填充到最大长度或等待所有请求完成。相反, 在每次迭代中, 已完成的请求会被及时过滤掉, 新的请求会根据可用的 GPU 动态添加到批次中

memory. As a result, individual requests can be processed and returned to the user more quickly, significantly reducing queuing time for new requests and improving overall system throughput.

For a typical implementation in popular frameworks, at a single decode iteration, the latest tokens of all current decoding requests are batched together for inference, and for the attention operation, a customized operator additionally accepts a list of the current request IDs and a mapping table (maintained by the memory management component) that associates these request IDs with their corresponding KV cache locations in the KV cache memory pool. The attention operator leverages this information to perform non-contiguous memory accesses within the KV cache memory pool, enabling it to compute attention operations for requests of varying sequence lengths. As for the linear operation in the LLM (which accounts for the vast majority of the model’s weights), since it is independent among the tokens (the same weight is applied uniformly across all tokens), it can be performed directly as a single matrix multiplication for the batched tokens.

2.4 Request Scheduler

Current LLM frameworks control the continuous batching process through a request scheduler. The scheduler filters out completed requests based on the running state of the requests in the running batch and decides whether to add queued requests to the running batch based on the available system memory resources. Due to the unpredictable and highly varying output lengths of LLM requests, current LLM frameworks use either conservative or aggressive schedulers for request scheduling. A conservative scheduler estimates memory usage based on the sum of request input lengths and the max_{new_tokens} , which can significantly limit the number of requests processed simultaneously, thus reducing throughput. On the other hand, an aggressive scheduler, which disregards memory occupancy from request outputs and batches requests solely based on input lengths, may lead to excessive request evictions due to insufficient memory during inference. This can severely break SLA constraints and negatively impact the user experience.

2.5 Challenges for Improving Goodput with SLA Guarantee

A service-level agreement (SLA) is a contract between a service provider and its customers that documents what services the provider furnishes and defines the service standards the provider is obligated to meet [38]. Systems that cannot meet the SLA guarantee lead to poor user experience. The key SLA metrics for LLM service include *Time To First Token* (TTFT), *Time Per Output Token* (TPOT), and *Max Time Per Output Token* (MTPOT). TTFT measures the time it takes for users to receive the initially generated token. TPOT indicates the interval between each token received by users. MTPOT

represents the maximum interval for users to receive each token, which is the maximum of TPOTs of a given request. Services that can guarantee SLA metrics for 99% of requests can always be seen as stable and available. Throughput under SLA is considered effective throughput, also known as goodput.

Achieving high system goodput remains a significant challenge due to the difficulty of estimating how scheduling new requests affects the memory requirements of the running batch. First, calculating the memory demand of a running batch is inherently challenging. Simply summing the input lengths of all requests or combining the input lengths with the maximum output length is not suitable because of the dynamic nature of request arrivals and removals. Second, predicting the output length of requests is also difficult, necessitating a precise yet low-overhead estimation approach. This is why current frameworks often compromise by opting for either aggressive or conservative schedulers.

3 Past-Future Request Scheduler

In this paper, we propose a novel Past-Future scheduler for LLM serving, which consists of the components of predicting output length distribution and estimating future required memory to achieve the precise scheduling of requests under SLA guarantees. The proposed scheduler can effectively address the challenges faced by existing request schedulers (discussed in Section 2.5)

3.1 Overview

As shown in Figure 2, the proposed Past-Future scheduler consists of two steps: predicting output length distribution and estimating future required memory. Specifically, at time t , the system has a running batch B with k requests $\{S^1, \dots, S^k\}$ and a request queue Q with $n-k$ waiting requests $\{S^{k+1}, \dots, S^n\}$. The proposed Past-Future scheduler will first predict the output length distribution for waiting requests in Q and update the predicted output lengths for requests in the running batch based on the distribution of historical request output lengths. Then, it will calculate the memory usage at each future time point after adding the waiting requests to the running batch, determine the future required memory, and schedule the requests based on the currently available memory in the system. With such an approach, it can flexibly schedule requests at the optimal time, achieving a reasonable trade-off between request queuing time and frequent request evictions, therefore maximizing system goodput. In the subsequent sections, we elaborate on how our scheduler predicts output length distribution (Section 3.2) and estimates future required memory (Section 3.3).

3.2 Predicting Output Length Distribution

To determine future memory requirements, we need to know the output length of the current requests. One approach is

记忆。因此，可以更快地处理单个请求并将其返回给用户，从而显着减少新请求的排队时间并提高整体系统吞吐量。

对于流行框架中的典型实现，在单次解码迭代中，所有当前解码请求的最新令牌被批处理在一起进行推理，并且对于注意力操作，定制的运算符另外接受当前请求ID的列表和映射表（由内存管理组件维护），该映射表将这些请求ID与KV缓存内存池中相应的KV缓存位置相关联。注意力运算符利用此信息在KV缓存内存池内执行非连续内存访问，使其能够针对不同序列长度的请求计算注意力操作。至于LLM中的线性运算（占模型权重的绝大多数），由于它在token之间是独立的（相同的权重均匀地应用于所有token），因此它可以直接作为批量token的单个矩阵乘法来执行。

2.4 请求调度器

当前的LLM框架通过请求调度程序控制连续批处理过程。调度器根据运行批次中请求的运行状态过滤掉已完成的请求，并根据可用的系统内存资源决定是否将排队请求添加到运行批次中。由于LLM请求的输出长度不可预测且变化很大，当前的LLM框架使用保守或激进的调度程序来进行请求调度。保守的调度程序根据请求输入长度和 $\max_{new_tokens}$ 的总和来估计内存使用量，这会显着限制同时处理的请求数量，从而降低吞吐量。另一方面，激进的调度程序会忽略请求输出的内存占用情况，仅根据输入长度对请求进行批处理，可能会因推理期间内存不足而导致过多的请求驱逐。这可能会严重破坏SLA限制并对用户体验产生负面影响。

2.5 通过SLA保证提高吞吐量的挑战

服务级别协议（SLA）是服务提供商与其客户之间的合同，记录了提供商提供的服务并定义了提供商有义务满足的服务标准[38]。无法满足SLA保证的系统会导致用户体验不佳。LLM服务的关键SLA指标包括首次令牌时间（TTFT）、每个输出令牌时间（TPOT）和每个输出令牌最大时间（MTPOT）。TTFT测量用户收到最初生成的令牌所需的时间。TPOT表示用户收到每个令牌之间的时间间隔。MTPOT

表示用户接收每个token的最大间隔，即给定请求的TPO T的最大值。能够保证99%请求的SLA指标的服务始终可以被视为稳定且可用。SLA下的吞吐量被认为是有效吞吐量，也称为有效吞吐量。

由于很难估计调度新请求如何影响正在运行的批处理的内存需求，因此实现高系统吞吐量仍然是一个重大挑战。首先，计算正在运行的批处理的内存需求本质上是具有挑战性的。由于请求到达和移除的动态特性，简单地将所有请求的输入长度相加或将输入长度与最大输出长度相结合是不合适的。其次，预测请求的输出长度也很困难，需要一种精确且低开销的估计方法。这就是为什么当前的框架经常通过选择激进或保守的调度程序来妥协。

3 过去未来请求调度程序

在本文中，我们提出了一种用于LLM服务的新颖的过去未来调度器，它由预测输出长度分布和估计未来所需内存的组件组成，以实现SLA保证下请求的精确调度。所提出的调度器可以有效地解决现有请求调度器所面临的挑战（在2.5节中讨论）

3.1 概述

如图2所示，所提出的过去-未来调度器由两个步骤组成：预测输出长度分布和估计未来所需的内存。具体来说，在时间 t ，系统有一个正在运行的批处理 B （包含 k 个请求 $\{S^1, \dots, S^k\}$ ）和一个请求队列 Q （包含 $n-k$ 个等待请求 $\{S^{k+1}, \dots, S^n\}$ ）。

所提出的过去未来调度程序将首先预测 Q 中等待请求的输出长度分布，并根据历史请求输出长度的分布更新正在运行的批次中请求的预测输出长度。然后，它将等待的请求添加到正在运行的批次中后，计算未来每个时间点的内存使用情况，确定未来所需的内存，并根据系统中当前可用的内存来调度请求。通过这种方法，它可以在最佳时间灵活地调度请求，在请求排队时间和频繁的请求驱逐之间实现合理的权衡，从而最大化系统的吞吐量。在后续部分中，我们详细说明调度程序如何预测输出长度分布（第3.2节）并估计未来所需的内存（第3.3节）。

3.2 预测输出长度分布

为了确定未来的内存需求，我们需要知道当前请求的输出长度。一种方法是

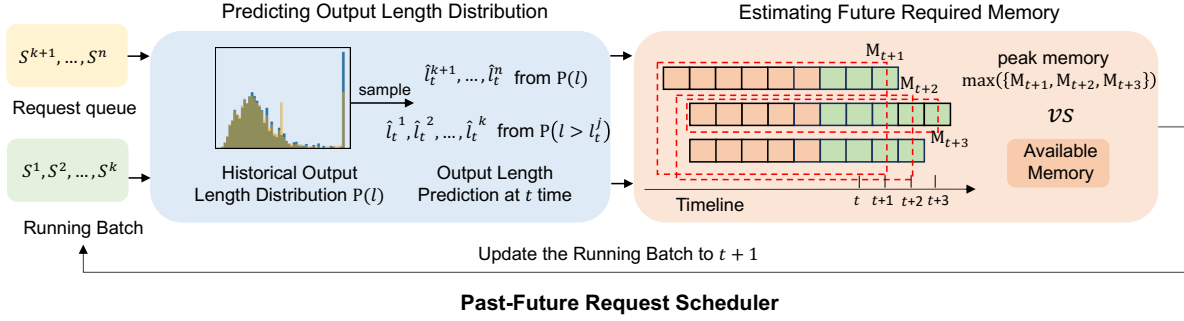


Figure 2. The overview of the proposed Past-Future scheduler. At time t , the service includes a running batch with k requests and a request queue with $n - k$ requests. The proposed scheduler will first predict the most probable output length for queued requests and update the predicted output length of each request in the running batch based on the historical distribution of request output lengths and their output lengths at time t . Then it calculates the memory demand of the running batch at each future time if the requests are added at time t , and determines whether to schedule the requests at time t based on whether any evictions would occur.

to use the maximum output length of the request, but since the LLM may end the output via EOS token at any time on its own, the actual output length may be much smaller than the maximum output length, leading to memory wastage. Therefore, accurately predicting the actual output length is essential for our scheduler.

However, predicting request output length is a highly challenging task. Existing prediction methods not only add extra inference overhead but also lack generalizability, making them impractical for real-world deployment. To address the issue, we first simplify the task to predict the output length distribution for all current requests, instead of seeking to accurately predict individual requests. After that, we can improve the prediction accuracy for individual requests by updating the prediction for each request on the fly based on the request’s already decoded length at runtime, allowing it to work with our module to estimate future required memory (Section 3.3) in order to achieve the high memory utilization scheduling without frequent request evictions.

For the output length distribution prediction, we analyzed multiple trace datasets from online LLM services including chat, code completion, and general API service, which leads to an interesting observation: *the distribution of request output lengths within a certain window size is consistently stable for some services (usually a single service or task type), whereas the distribution of output lengths for other hybrid task or API services may vary over a long time period (hours or days), but still tends to be stable within a short time period (minutes)*. This means that both types of service requests in two adjacent sliding time windows are likely to have similar output length distributions. Therefore, we can effectively estimate the output length for requests of the currently running batch based on the historical distribution of recently finished requests.

Figure 3 shows the cosine similarity between the output length distribution at different time windows on multiple

online service requests trace datasets. BurstGPT [34] contains trace data for over 1,400,000 requests made to ChatGPT and GPT-4. The requests are divided into two categories: (a) online conversation service requests, and (b) API service requests, which are the two most common invocation methods for LLM services. We have also validated our approach on more datasets from our in-house services (c, d, e) and the Mooncake [25] trace from Moonshot (f). We find that for a single end-user service, including dialog (a, c, d, f) or code completion (e), its overall distribution is less varying. For API services (b), on the other hand, its overall distribution is more varying as it usually mixes multiple service types (to the best of our knowledge, there are no more publicly available trace datasets for LLM API services or hybrid tasks yet). However, for all datasets, the adjacent time windows (the diagonal pattern) have similar distributions. We believe that the services in the above datasets cover the vast majority of current scenarios of LLM service, and thus this distribution similarity of adjacent time windows is universal.

In practice, the size of the historical request window (the preceding window in adjacent windows) can be freely adjusted, whereas the size of the current request window (the succeeding window in adjacent windows) is determined by the current batch size. To evaluate the effectiveness under various workload conditions, we conducted experiments with different combinations of window sizes on the BurstGPT dataset. The conversation logs from the BurstGPT dataset are shown in red, and the API logs are shown in blue, with different brightnesses of a color representing various running window sizes, as illustrated in Figure 4. The results show the distribution similarity of adjacent windows (diagonal) is maintained at a high level across multiple running window sizes. Variations in the historical window size exhibit distinct trends for different request types, but a size of 1,000 balances both types effectively. For simplicity, we have

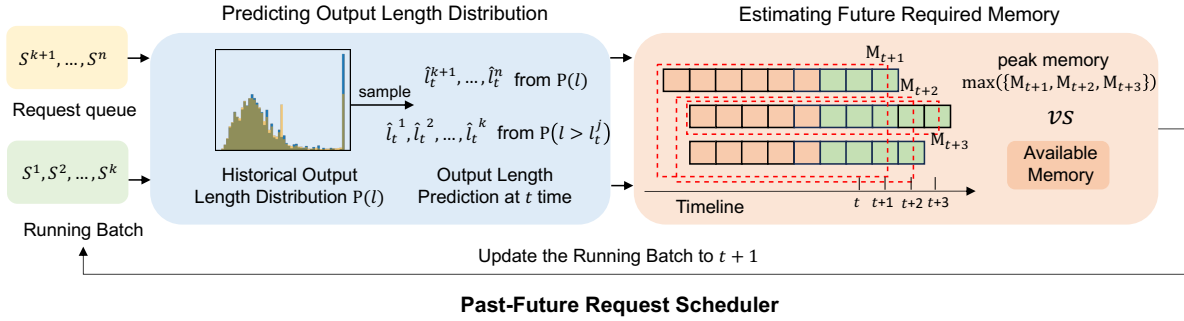


图 2. 所提出的过去未来调度程序的概述。在时间 t ，服务包括一个正在运行的包含 k 请求的批处理和一个包含 $n - k$ 请求的请求队列。建议的调度程序将首先预测排队请求的最可能输出长度，并根据请求输出长度的历史分布及其在时间 t 的输出长度来更新正在运行的批次中每个请求的预测输出长度。然后，如果在时间 t 添加请求，它会计算运行批次在未来每个时间的内存需求，并根据是否会发生驱逐来确定是否在时间 t 调度请求。

使用请求的最大输出长度，但由于LLM可能随时自行结束通过EOS token的输出，因此实际输出长度可能远小于最大输出长度，导致内存浪费。因此，准确预测实际输出长度对于我们的调度程序至关重要。

然而，预测请求输出长度是一项极具挑战性的任务。现有的预测方法不仅增加了额外的推理开销，而且缺乏通用性，使得它们对于现实世界的部署来说不切实际。为了解决这个问题，我们首先简化任务来预测所有当前请求的输出长度分布，而不是寻求准确预测单个请求。之后，我们可以根据请求在运行时已解码的长度动态更新每个请求的预测，从而提高各个请求的预测准确性，使其与我们的模块一起估计未来所需的内存（第3.3节），以实现高内存利用率调度，而无需频繁的请求驱逐。

对于输出长度分布预测，我们分析了来自在线LLM服务（包括聊天、代码完成和通用API服务）的多个跟踪数据集，这导致了一个有趣的观察：对于某些服务（通常是单个服务或任务类型），特定窗口大小内的请求输出长度的分布始终稳定，而其他混合任务或API服务的输出长度分布可能在很长一段时间内（几小时或几天）变化，但在短时间内（分钟）内仍然趋于稳定。这意味着两个相邻滑动时间窗口中的两种类型的服务请求很可能具有相似的输出长度分布。因此，我们可以根据最近完成的请求的历史分布来有效地估计当前运行批次的请求的输出长度。

图3显示了多个不同时间窗口的输出长度分布之间的余弦相似度

在线服务请求跟踪数据集。BurstGPT [34] 包含对 ChatGPT 和 GPT-4 发出的超过 1,400,000 个请求的跟踪数据。请求分为两类：（a）在线会话服务请求，（b）API服务请求，这是LLM服务最常见的两种调用方式。我们还在来自我们内部服务的更多数据集（c、d、e）和来自 Moonshot 的 Mooncake [25] 跟踪（f）上验证了我们的方法。我们发现，对于单个最终用户服务，包括对话（a、c、d、f）或代码完成（e），其总体分布变化较小。另一方面，对于 API 服务（b），其总体分布更加多样化，因为它通常混合了多种服务类型（据我们所知，目前还没有更多公开可用的 LLM API 服务或混合任务的跟踪数据集）。然而，对于所有数据集，相邻时间窗口（对角线模式）具有相似的分布。我们认为，上述数据集服务中的服务涵盖了目前LLM服务的绝大多数场景，因此相邻时间窗口的这种分布相似性具有普遍性。

实际上，历史请求窗口（相邻窗口中的前一个窗口）的大小可以自由调整，而当前请求窗口（相邻窗口中的后一个窗口）的大小由当前批量大小决定。为了评估各种工作负载条件下的有效性，我们在 BurstGPT 数据集上进行了不同窗口大小组合的实验。BurstGPT 数据集的对话日志以红色显示，API 日志以蓝色显示，不同的颜色亮度代表不同的运行窗口大小，如图 4 所示。结果表明，在多个运行窗口大小中，相邻窗口（对角线）的分布相似性保持在较高水平。历史窗口大小的变化对于不同的请求类型表现出不同的趋势，但是 1,000 的大小可以有效地平衡这两种类型。为了简单起见，我们有

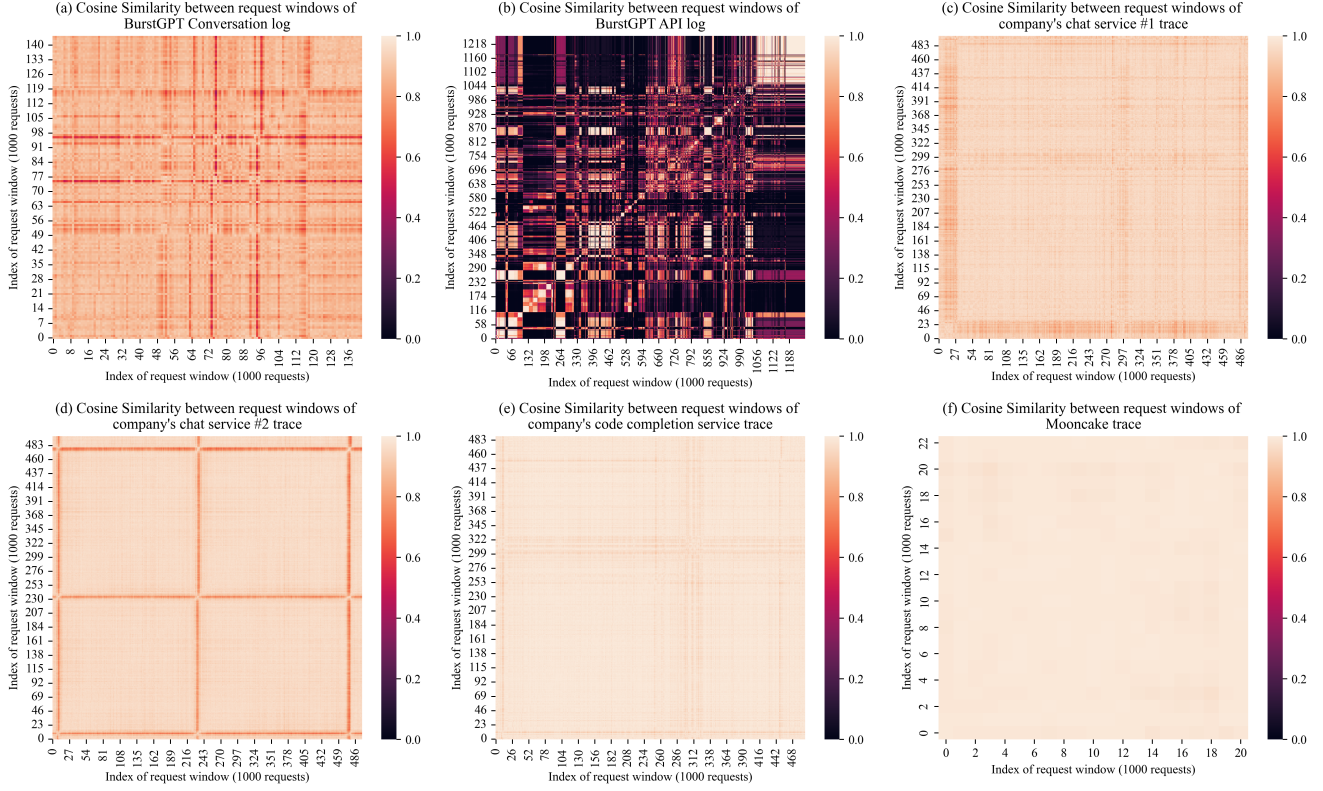


Figure 3. Cosine similarity of output length distributions (higher values indicate higher similarity) between the different partitioned time windows (1000 requests, no overlap) of multiple request trace dataset, points indicating the overlapped request window are excluded. The adjacent time windows (the diagonal pattern) have similar distributions (some dataset has similar distributions globally).

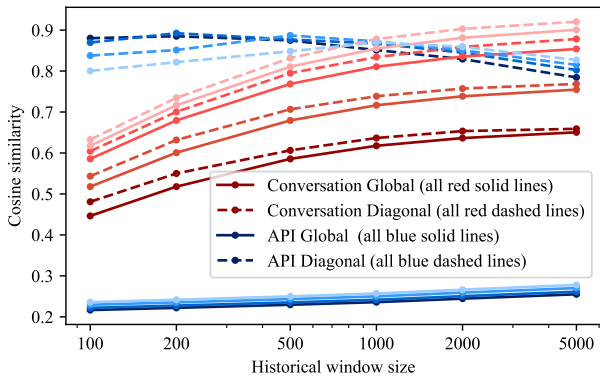


Figure 4. Average cosine similarity on global or diagonal line under different historical (x-axis) and running request windows size (in different brightnesses of a color, corresponding to the number markers on the right) in the BurstGPT dataset.

adopted this setting uniformly in subsequent experiments, as further tuning yields negligible improvements to scheduling effectiveness.

Based on the above observations, we introduce an efficient, parameter-free prediction method, for predicting output length distribution, which estimates and dynamically updates the output lengths for each request in the current running batch at each time step based on the historical distribution of recently finished request output lengths.

Specifically, the proposed method utilizes the probability distribution of historical request output lengths to dynamically predict the output length of current requests at each inference step. During serving, it records the actual output lengths of historical requests from adjacent time windows, denoted $L_h = \{l_h^0, l_h^1, \dots, l_h^w\}$ where w is the window size. Thus the probability distribution can be described as:

$$P(l) = C(l, L_h) / w \quad (1)$$

where $C(l, L_h)$ is a counter function, which returns the number of elements in L_h equal to l .

Therefore, as shown in Figure 2, at time step t , before estimating future required memory, the proposed scheduler will first sample a $\hat{l}_t^i$ for each request S^i in the request queue $S_q = \{S^{k+1}, \dots, S^n\}$ from $P(l)$ and update the predicted length $L_{t-1} = \{\hat{l}_{t-1}^1, \hat{l}_{t-1}^2, \dots\}$.

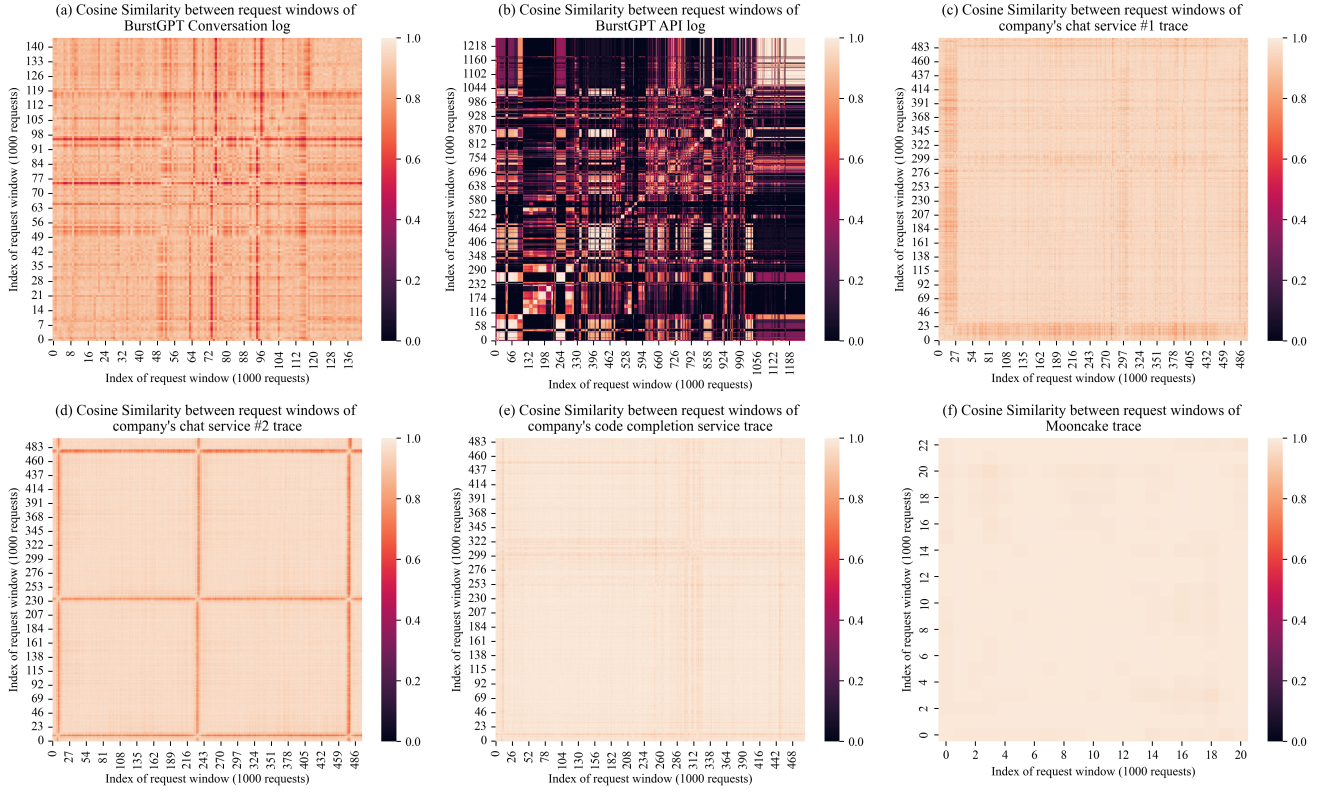


图 3. 多个请求跟踪数据集的不同分区时间窗口（1000 个请求，无重叠）之间的输出长度分布的余弦相似度（值越高表示相似度越高），指示重叠请求窗口的点被排除。相邻时间窗口（对角线模式）具有相似分布（某些数据集在全球范围内具有相似分布）。

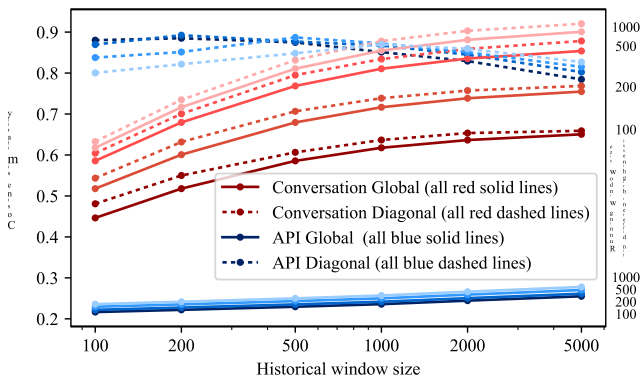


图 4. BurstGPT 数据集中不同历史（x 轴）和运行请求窗口大小（不同颜色亮度，对应右侧数字标记）下全局或对角线上的平均余弦相似度。

在后续的实验中统一采用了这个设置，因为进一步的调整对调度效率的提高可以忽略不计。

基于上述观察，我们引入了一种高效、无参数的预测方法，用于预测输出长度分布，该方法根据最近完成的请求输出长度的历史分布，在每个时间步估计并动态更新当前运行批次中每个请求的输出长度。

具体来说，所提出的方法利用历史请求输出长度的概率分布来动态预测每个推理步骤当前请求的输出长度。在服务期间，它记录相邻时间窗口的历史请求的实际输出长度，表示为 $L_h = \{l_h^0, l_h^1, \dots, l_h^w\}$ ，其中 w 是窗口大小。因此概率分布可以描述为：

$$P(l) = C(l, L_h) / w \quad (1)$$

其中 $C(l, L_h)$ 是一个计数器函数，它返回 L_h 中等于 l 的元素数量。

因此，如图 2 所示，在时间步 t 处，在估计未来所需内存之前，建议的调度程序将首先从 $P(l)$ 中为请求队列 $S_q = \{s^{k+1}, \dots, s^n\}$ 中的每个请求 s^i 采样一个 $\hat{l}_t^i$ ，并更新预测长度 $L_{t-1} = \{\hat{l}_{t-1}^1, \hat{l}_{t-1}^2, \dots\}$ 。

$\dots, \hat{l}_{t-1}^k\}$ by resampling $\hat{l}_t^j$ for each request S^j in the running batch $S = \{S^0, S^1, \dots, S^n\}$ from $P(l > l_{t-1}^j)$ to get a new predicted output length $L_t = \{\hat{l}_t^1, \hat{l}_t^2, \dots, \hat{l}_t^k\}$.

By dynamically updating the output length of the request at each step, the proposed scheduler can ensure that the predicted values are always approaching the real output length distribution and the predicted length is larger than the actual output length as much as possible. Therefore, it can not only accommodate as many requests as possible, but also ensures sufficient memory to guarantee the completion of requests, thereby avoiding request eviction. Additionally, in contrast to some existing methods that obtain the output length by classifying the input content or fine-tuning the LLM, our method has very high generalizability (model-independent) and without incurring any additional inference overhead. Algorithm 1 describes the details of our proposed scheduler to handle new requests at time t .

Algorithm 1 Past-Future Scheduler

Input: historical output lengths L_h , request queue S_q , running batch S with the number of generated tokens $\{l_t^1, l_t^2, \dots, l_t^k\}$ at time $t - 1$, system memory capacity M .
Output: the updated running batch S

- 1: Build the probability distribution of output length $P(l)$ with L_h , via Equation 1
- 2: Initialize the predicted output length $L_t = \{\}$
- 3: **for** l_t^i in $\{l_t^1, l_t^2, \dots, l_t^k\}$ **do**
- 4: Sample $\hat{l}_t^i$ from $P(l > l_t^i)$
- 5: $L_t \leftarrow L_t \cup \{\hat{l}_t^i\}$
- 6: **end for**
- 7: **for** S^j in S_q **do**
- 8: Sample $\hat{l}_t^j$ from $P(l)$
- 9: $L_t \leftarrow L_t \cup \{\hat{l}_t^j\}$
- 10: Calculate future required memory M_t^* with L_t (and other known information), via Equation 2 to 4
- 11: **if** $M_t^* \leq M$ **then**
- 12: $S \leftarrow S \cup \{S^j\}$
- 13: **else**
- 14: Return S
- 15: **end if**
- 16: **end for**

3.3 Estimating Future Required Memory

After predicting the output length of each request, the memory demand of the running batch can be calculated. Estimating the memory demand for the entire running batch remains challenging due to the varying entry and completion times of requests. Current schedulers, which statically estimate memory usage, can only account for memory release when a request is completed. This results in either low memory utilization with a conservative approach or frequent request evictions with an aggressive approach. As shown in

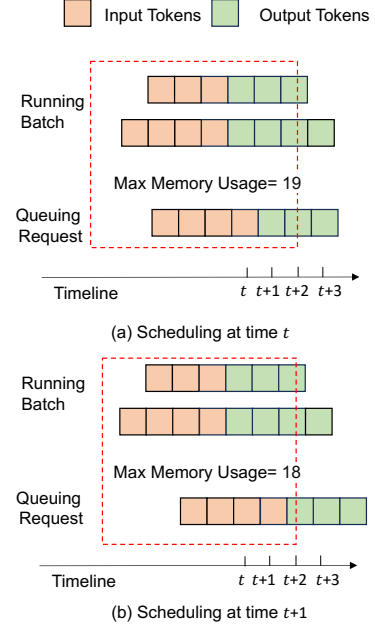


Figure 5. Memory demand of a running batch when the queuing request is scheduled at different time steps.

Figure 5, even if no request is completed, adding the same request into the running batch at different time steps leads to varying memory demands. Therefore, our scheduler incorporates a novel method to consider the future process when scheduling new queued requests. By anticipating the memory resources released by future completed requests, it dynamically schedules queued requests at optimal time points, minimizing queuing delays while preventing frequent evictions during inference. This ensures high goodput while maintaining SLA guarantees.

Specifically, given a running batch $S = \{S^1, S^2, \dots, S^k\}$ with a predicted output length $L_t = \{\hat{l}_t^1, \hat{l}_t^2, \dots, \hat{l}_t^k\}$ at step t , we can accurately calculate the memory requirement for inference of each request, which is the sum of the input length and output length. However, the memory required to complete the entire running batch of requests is not simply the sum of the memory needed for all individual requests, since each request in S will finish at different time steps and dynamically release the memory resource. Simply using the sum of the memory of all requests to determine whether the system has sufficient memory to complete the currently running batch successfully leads to low memory utilization. In contrast, the required memory at future time points is the minimum memory requirement to complete the current batch.

As illustrated on the right side of Figure 2, we note that the future required memory will inevitably occur at the moment a request ends. If no request ends at time point t , the GPU memory usage of the running batch at t must be less than that at $t + 1$. At the $t + 2$ time point, the request with

$\dots, \hat{l}_{t-1}^k\}$, 通过对来自 $P(l > l_{t-1}^j)$ 的正在运行的批次 $S = \{S^0, S^1, \dots, S^m\}$ 中的每个请求 S^j 重新采样 $\hat{l}_t^j$, 以获得新的预测输出长度 $L_t = \{\hat{l}_t^1, \hat{l}_t^2, \dots, \hat{l}_t^k\}$ 。通过在每一步动态更新请求的输出长度, 所提出的调度器可以确保预测值始终接近真实输出长度分布, 并且预测长度尽可能大于实际输出长度。因此, 它不仅可以容纳尽可能多的请求, 而且可以保证足够的内存来保证请求的完成, 从而避免请求被驱逐。此外, 与一些现有的通过对输入内容进行分类或微调LLM来获得输出长度的方法相比, 我们的方法具有非常高的通用性 (与模型无关), 并且不会产生任何额外的推理开销。算法 1 描述了我们建议的调度程序的详细信息, 该调度程序用于在时间 t 处理新请求。

Algorithm 1 Past-Future Scheduler

Input: historical output lengths L_h , request queue S_q , running batch S with the number of generated tokens $\{l_t^1, l_t^2, \dots, l_t^k\}$ at time $t - 1$, system memory capacity M .
Output: the updated running batch S

- 1: Build the probability distribution of output length $P(l)$ with L_h , via Equation 1
- 2: Initialize the predicted output length $L_t = \{\}$
- 3: **for** l_t^i in $\{l_t^1, l_t^2, \dots, l_t^k\}$ **do**
- 4: Sample $\hat{l}_t^i$ from $P(l > l_t^i)$
- 5: $L_t \leftarrow L_t \cup \{\hat{l}_t^i\}$
- 6: **end for**
- 7: **for** S^j in S_q **do**
- 8: Sample $\hat{l}_t^j$ from $P(l)$
- 9: $L_t \leftarrow L_t \cup \{\hat{l}_t^j\}$
- 10: Calculate future required memory M_t^* with L_t (and other known information), via Equation 2 to 4
- 11: **if** $M_t^* \leq M$ **then**
- 12: $S \leftarrow S \cup \{S^j\}$
- 13: **else**
- 14: Return S
- 15: **end if**
- 16: **end for**

3.3 估计未来所需的内存

在预测每个请求的输出长度后, 可以计算正在运行的批处理的内存需求。由于请求的进入和完成时间各不相同, 估计整个运行批次的内存需求仍然具有挑战性。当前的调度程序静态地估计内存使用情况, 只能在请求完成时考虑内存释放。这会导致保守方法的内存利用率较低或激进方法的频繁请求驱逐。如图所示

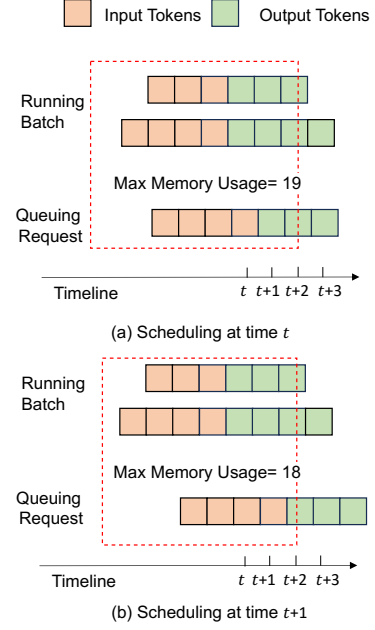


图 5. 当排队请求安排在不同的时间步长时, 正在运行的批处理的内存需求。

如图 5 所示, 即使没有请求完成, 在不同的时间步骤将相同的请求添加到正在运行的批处理中也会导致不同的内存需求。因此, 我们的调度程序采用了一种新颖的方法来在调度新的排队请求时考虑未来的进程。通过预测未来完成的请求释放的内存资源, 它可以在最佳时间点动态调度排队的请求, 最大限度地减少排队延迟, 同时防止推理期间频繁驱逐。这可确保高吞吐量, 同时维持 SLA 保证。

具体来说, 给定一个正在运行的批次 $S = \{S^1, S^2, \dots, S^k\}$, 在步骤 t 处具有预测的输出长度 $L_t = \{\hat{l}_t^1, \hat{l}_t^2, \dots, \hat{l}_t^k\}$, 我们可以准确计算每个请求的推理所需的内存需求, 即输入长度和输出长度的总和。然而, 完成整个正在运行的请求批次所需的内存不仅仅是所有单独请求所需内存的总和, 因为 S 中的每个请求将在不同的时间步完成并动态释放内存资源。简单地使用所有请求的内存总和来确定系统是否有足够的内存来成功完成当前正在运行的批处理会导致内存利用率较低。相反, 未来时间点所需的内存是完成当前批次的最小内存要求。

如图 2 右侧所示, 我们注意到未来所需的内存将不可避免地发生在请求结束的那一刻。如果在时间点 t 没有请求结束, 则 t 运行批次的 GPU 内存使用量必须小于 $t+1$ 时的 GPU 内存使用量。在 $t+2$ 时间点, 请求

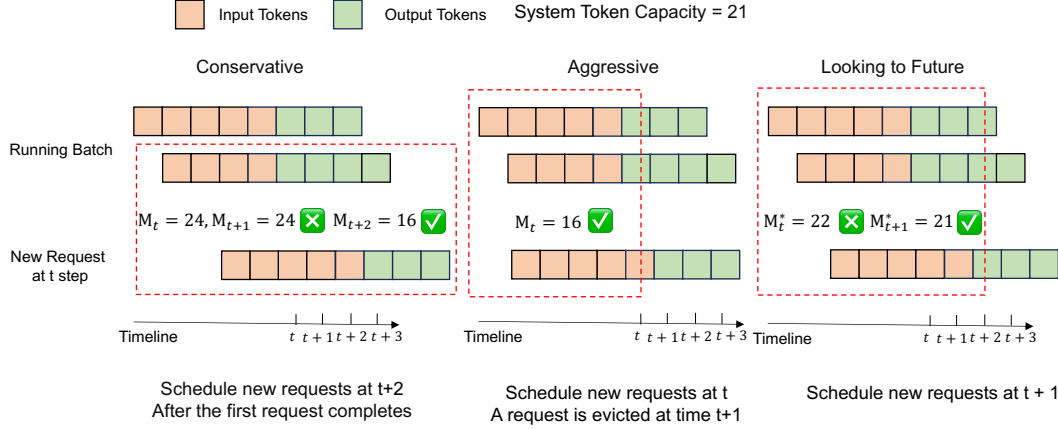


Figure 6. Behavior comparison of different schedulers when handling new requests at time t . It shows that our method can introduce requests at the appropriate time, maintaining high memory utilization while avoiding harmful request evictions.

the shortest output completes and releases the memory it occupies, reducing the memory occupancy of service to M_{t+2} . At $t+3$ time, this leaves only one request that needs to be processed, which requires the memory of M_{t+3} . Therefore, only $\max(M_{t+1}, M_{t+2}, M_{t+3})$ space is needed to complete the inference of requests in the batch. To calculate memory usage when each request is completed, the requests are first sorted in descending order based on the estimated remaining generation length.

$$\{(l_p^i + l_t^i, \hat{l}_t^i - l_t^i) | S^i \in S\} \quad (2)$$

where $\hat{l}_t^i - l_t^i \geq \hat{l}_t^{i+1} - l_t^{i+1}$, $i = 1, 2, \dots, k$

where l_p^i is the input length of request S^i , l_t^i is the number of generated tokens of request S^i at time t , $\hat{l}_t^i$ is the predicted output length of request S^i , k is the current batch size. Then, we can calculate the memory occupancy of the system when the S^i request finishes:

$$M_i = (\sum_{j=1}^i \{l_p^j + l_t^j\}) + (\hat{l}_t^i - l_t^i) * i \quad (3)$$

Finally, we can determine that the future required memory to complete Batch S is:

$$M^* = \max(\{M_i | i = 1, 2, \dots, k\}) \quad (4)$$

Figure 6 illustrates the differences between the proposed scheduler and the conservative or aggressive schedulers currently used by existing frameworks when handling new requests. Assume that the system has a total capacity of 21 tokens. The conservative scheduler only adds new requests when it is ensured that the system memory resources exceed the total resources required by all requests (at time $t+2$, the first request has been completed). It not only results in high response latency, but also leads to low utilization of hardware resources. The aggressive scheduler, on the other hand,

immediately adds a request if it finds sufficient system memory to handle the input (at time t). However, as inference progresses, it often discovers that the system resources are insufficient (at time $t+2$, $M_{t+2} = 22 > 21$), necessitating the eviction of a request to ensure that inference can continue (which requires request re-queuing and recomputation). In contrast, our scheduler evaluates the impact of adding requests at different times on the future required memory usage of the system. Consequently, it schedules requests at the most appropriate time (at $t+1$), allowing requests to wait for a few decoding steps to avoid eviction. It ensures better utilization of system resources while reducing the risk of request evictions.

4 Implementation

To demonstrate the effectiveness of the proposed Past-Future scheduler, we developed a high-performance LLM serving framework, LightLLM. LightLLM is a Python-based framework for LLM inference and serving, designed to be lightweight, easily scalable, and high-performance. It realizes multi-process asynchronous collaboration, parallelizing request pre-processing, scheduling with model inference, and post-processing. The front end of LightLLM is built using FastAPI, while the backend inference relies primarily on PyTorch [23] and OpenAI Triton [31]. A wide range of models has already been supported, including BLOOM [39], Llama [32], Llama2 [32], StarCoder [14], ChatGLM2-6b [7, 41], Qwen [2], Baichuan [4], Baichuan2 [4], InternLM [30], and Yi [29]. Moreover, LightLLM also supports the deployment of multi-modal large language models, such as LLaVA [16] and Qwen-VL-Chat [3]. The complete code has been [open-sourced](#).

Scheduler workflow. We implemented the Past-Future scheduler using parallel computation on GPUs in LightLLM,

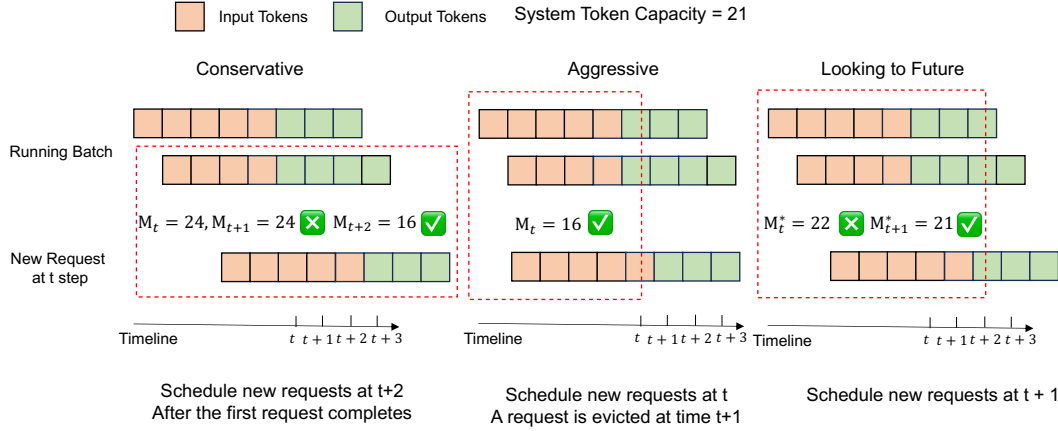


图 6. 不同调度程序在时间 t 处理新请求时的行为比较。它表明我们的方法c在适当的时间引入请求，保持高内存利用率，同时避免有害的请求驱逐

一个
S。

最短的输出完成并释放其占用的内存，将service的内存占用降低到 M_{t+2} 。在 $t+3$ 次，只剩下一个请求需要处理，这需要 M_{t+3} 的内存。因此，只需要 $\max(M_{t+1}, M_{t+2}, M_{t+3})$ 空间即可完成批量请求的推理。为了计算每个请求完成时的内存使用情况，首先根据估计的剩余生成长度按降序对请求进行排序。

$$\{(l_p^i + l_t^i, \hat{l}_t^i - l_t^i) | S^i \in S\} \quad (2)$$

where $\hat{l}_t^i - l_t^i \geq \hat{l}_t^{i+1} - l_t^{i+1}, i = 1, 2, \dots, k$

其中 l_p^i 是请求 S^i 的输入长度， l_t^i 是请求 S^i 在时间 t 生成的令牌数量， $\hat{l}_t^i$ 是请求 S^i 的预测输出长度， k 是当前批量大小。然后，我们可以计算出 S^i 请求完成时系统的内存占用情况：

$$M_i = (\sum_{j=1}^i \{l_p^j + l_t^j\}) + (\hat{l}_t^i - l_t^i) * i \quad (3)$$

最后，我们可以确定未来完成 Batch S 所需的内存为：

$$M^* = \max(\{M_i | i = 1, 2, \dots, k\}) \quad (4)$$

图 6 说明了所提出的调度程序与现有框架在处理新请求时当前使用的保守或激进调度程序之间的差异。假设系统总容量为21个代币。保守调度器只有在确保系统内存资源超过所有请求所需的总资源时才添加新的请求（在时间 $t+2$ ，第一个请求已经完成）。它不仅导致响应延迟高，而且导致硬件资源利用率低。另一方面，激进的调度程序

如果发现足够的系统内存来处理输入（在时间 t ），则立即添加请求。然而，随着推理的进行，经常会发现系统资源不足（在时间 $t+2$ ， $M_{t+2} = 22 > 21$ ），需要驱逐请求以确保推理能够继续（这需要请求重新排队和重新计算）。相反，我们的调度程序评估在不同时间添加请求对系统未来所需内存使用的影响。因此，它会在最合适的时间（在 $t+1$ ）安排请求，允许请求等待一些解码步骤以避免驱逐。它确保更好地利用系统资源，同时降低请求驱逐的风险。

4 实施

为了证明过去未来调度程序的有效性，我们开发了一个高性能的 LLM 服务框架 LightLLM。LightLLM 是一个基于 Python 的 LLM 推理和服务框架，旨在轻量级、易于扩展和高性能。它实现了多进程异步协作、并行化请求预处理、模型推理调度和后处理。LightLLM 的前端是使用 FastAPI 构建的，而后端推理主要依赖于 PyTorch [23] 和 OpenAI Triton [31]。已经支持多种模型，包括 BLOOM [39]、Llama [32]

Llama2 [32]、StarCoder [14]、ChatGLM2-6b [7, 41]、Qwen [2]、Baichuan [4]、Baichuan2 [4]、InternLM [30] 和 Yi [29]。此外，LightLLM 还支持多模态大语言模型的部署，例如 LLaVA [16] 和 Qwen-VL-Chat [3]。完整的代码已经开源。

调度程序工作流程。我们在 LightLLM 中使用 GPU 上的并行计算实现了过去未来调度程序，

which has a low overhead (less than 1% of LLM model inference time). We evaluated different historical request window sizes from hundreds to thousands on multiple datasets, which all worked well with our scheduler. Therefore, we chose 1,000 as an efficient and widely adopted setting. When the size of the running batch is low, the sampling prediction is repeated several times to improve accuracy. Moreover, at the initial stage of service startup, we initialize the output length distribution using the preset maximum output length, which can be updated quickly in a few minutes.

5 Evaluation

In this section, we provide a comprehensive evaluation of our framework across multiple models and various hardware platforms to prove its superiority over state-of-the-art LLM request schedulers.

5.1 Experimental Setup

Model. To demonstrate the effectiveness of our scheduler, we choose Llama2 [33], the most commonly used model in both industry and academia, for performance analysis. We evaluate models of three different scales, 7B, 13B, and 70B, to compare performance across different schedulers implemented in LightLLM. Additionally, we evaluate the performance improvement of our scheduler on multimodal tasks using popular models such as Qwen-VL-Chat [3] and LLaVA-1.5 [15].

Dataset. To evaluate the performance of different schedulers under varying output length distributions, we experiment three datasets with different distributions (*Distribution-1*, *-2*, *-3*) based on the request distribution of our real-world LLM service, with *input/output* length uniform distributions of *32-4k/2k-4k*, *3k-5k/3k-5k*, and *2k-4k/32-4k*. We also experiment with the ShareGPT-o1 dataset using the ShareGPT dataset and the OpenAI o1-preview API. By sending user questions from the ShareGPT dataset to the API, we collect output token counts, creating a request-length dataset reflective of the Chain of Thought model serving workloads. Additionally, for multimodal performance assessment, we adopt the validation set of the TextVQA [28] dataset, which contains 5,000 questions and 3,166 images.

SLA guarantee. A service that meets SLA guarantees is a usable service. Therefore, we mainly focus on the throughput comparison of LLM service frameworks equipped with different schedulers under SLA guarantees. Therefore, we only measure the throughput of requests that meet the SLA, which we refer to as goodput.

Common LLM applications such as chatbots use streaming output methods, where Time To First Token (TTFT) is the time from when the user sends a request to when the first output token is received, and Time Per Output Token (TPOT) is the delay between each subsequent output token. For the average TPOT, it is generally accepted that it needs

to be larger than the human reading speed, which is usually satisfied by existing frameworks. Equally important is the maximum TPOT in a request (MTPOT), which is one of the metrics we use subsequently. This is because as soon as there exists one high TPOT, the user will observe a noticeable output stall, while the average TPOT or even the P99 TPOT can hardly reflect this occurrence.

In the following experiments, we define the SLA guarantee based on the hardware capabilities and model sizes: ($TTFT < 10$ seconds, $MTPOT < 1.5$ seconds) for 7B/13B models, and ($TTFT < 15$ seconds, $MTPOT < 5$ seconds) for 70B model across all hardware.

5.2 Comparison with Different Schedulers

Figure 7 shows the goodput of different schedulers across various model sizes and datasets on NVIDIA A100-80G (1 GPU for 7B and 13B models, 4 GPUs with NVLink for 70B model). It can be observed that when there are few concurrent clients, the system memory load is low, resulting in the same goodput performance across different schedulers. As concurrency increases, conservative schedulers waste a significant amount of memory, causing many requests to be queued and leading to first-token latency breaking the $TTFT$ SLA limit, resulting in lower goodput compared to aggressive schedulers and the proposed Past-Future scheduler. As concurrency continues to rise to full system load, aggressive schedulers suffer from repeated evictions, causing requests to violate the $MTPOT$ SLA constraint, which leads to a decline in goodput. In contrast, our Past-Future scheduler accurately estimates the maximum memory demand of each running batch, ensuring that all scheduled requests are completed successfully. Thus, as the number of concurrent clients increases, the goodput eventually stabilizes, reaching the maximum that can be handled by the hardware under the given SLA constraints.

Moreover, it is worth noting that the schedulers exhibit different performances under varying input-output length distributions. In prefill-heavy datasets (where input length $>$ output length), such as *Distribution-3*, both the aggressive scheduler and our Past-Future scheduler achieve strong performance, surpassing the conservative scheduler. However, in decode-heavy datasets (where output length $>$ input length), such as *ShareGPT-o1* and *Distribution-1*, the aggressive scheduler, due to ignoring output memory requirements, leads to frequent eviction of requests, causing goodput to first increase and then decline. In contrast, our proposed scheduler ensures stable performance across various LLM scenarios, regardless of the input-output length distribution.

5.3 Ablation Study

In this part, we conduct an in-depth analysis of the Past-Future scheduler on decode-heavy *Distribution-1*, balanced *Distribution-2* and prefill-heavy *Distribution-3* from two perspectives: memory utilization and request eviction. Table 1

其开销较低（小于 LLM 模型推理时间的 1%）。我们在多个数据集上评估了从数百到数千的不同历史请求窗口大小，这些都与我们的调度程序配合良好。因此，我们选择 1,000 作为高效且广泛采用的设置。当运行批次的大小较低时，会重复多次采样预测以提高准确性。而且，在服务启动的初始阶段，我们使用预设的最大输出长度来初始化输出长度分布，可以在几分钟内快速更新。

5 评价

在本节中，我们跨多个模型和各种硬件平台对我们的框架进行全面评估，以证明其相对于最先进的 LLM 请求调度程序的优越性。

5.1 实验设置

模型。为了证明我们的调度程序的有效性，我们选择 LLaMA2 [33]（工业界和学术界最常用的模型）进行性能分析。我们评估了三种不同规模（7B、13B 和 70B）的模型，以比较 LightLLM 中实现的不同调度程序的性能。此外，我们使用 Qwen-VL-Chat [3] 和 LLaVA-1.5 [15] 等流行模型来评估调度程序在多模式任务上的性能改进。

数据集。为了评估不同调度器在不同输出长度分布下的性能，我们根据实际 LLM 服务的请求分布，实验了具有不同分布（Distribution-1、-2、-3）的三个数据集，其中输入/输出长度均匀分布为 32-4k/2k-4k、3k-5k/3k-5k 和 2k-4k/32-4k。我们还使用 ShareGPT 数据集和 OpenAI o1-preview API 对 ShareGPT-o1 数据集进行实验。通过将用户问题从 ShareGPT 数据集发送到 API，我们收集输出令牌计数，创建反映服务工作负载的思想链模型的请求长度数据集。此外，对于多模式性能评估，我们采用 TextVQA [28] 数据集的验证集，其中包含 5,000 个问题和 3,166 张图像。

SLA 保证。满足 SLA 保证的服务才是可用的服务。因此，我们主要关注 SLA 保证下配备不同调度器的 LLM 服务框架的吞吐量比较。因此，我们只测量满足 SLA 的请求的吞吐量，我们将其称为有效吞吐量。

聊天机器人等常见的 LLM 应用程序使用流式输出方法，其中第一个令牌的时间（TTFT）是从用户发送请求到收到第一个输出令牌的时间，每个输出令牌的时间（TPOT）是每个后续输出令牌之间的延迟。对于平均 TPOT，人们普遍认为它需要

大于人类的阅读速度，现有框架通常可以满足这一要求。同样重要的是请求中的最大 TPOT（MTPOT），这是我们随后使用的指标之一。这是因为一旦存在一个高 TPOT，用户就会观察到明显的输出失速，而平均 TPOT 甚至 P99 TPOT 很难反映这种情况的发生。

在以下实验中，我们根据硬件功能和模型大小定义 SLA 保证：对于 7B/13B 模型为（ $TTFT < 10$ 秒， $MTPOT < 1.5$ 秒），对于所有硬件的 70B 模型为（ $TTFT < 15$ 秒， $MTPOT < 5$ 秒）。

5.2 与不同调度器的比较

图 7 显示了 NVIDIA A100-80G 上不同模型大小和数据集的不同调度程序的吞吐量（7B 和 13B 模型使用 1 个 GPU，70B 模型使用 4 个带 NVLink 的 GPU）。可以看出，当并发客户端较少时，系统内存负载较低，从而导致不同调度程序具有相同的良好输出性能。随着并发性的增加，保守的调度程序会浪费大量内存，导致许多请求排队，并导致第一个令牌延迟突破 $TTFT$ SLA 限制，从而导致与激进的调度程序和建议的过去未来调度程序相比，吞吐量较低。随着并发继续上升到满系统负载，激进的调度程序会遭受重复驱逐，导致请求违反 $MTPOT$ SLA 约束，从而导致吞吐量下降。相比之下，我们的过去-未来调度程序准确地估计每个运行批次的最大内存需求，确保所有调度的请求都成功完成。因此，随着并发客户端数量的增加，吞吐量最终会稳定下来，达到给定 SLA 约束下硬件可以处理的最大值。

此外，值得注意的是，调度器在不同的输入输出长度分布下表现出不同的性能。在预填充密集的数据集中（其中输入长度 > 输出长度），例如 Distribution-3，积极的调度器和我们的过去未来调度器都实现了强大的性能，超越了保守的调度器。然而，在解码繁重的数据集中（其中输出长度 > 输入长度），例如 ShareGPT-o1 和 Distribution-1，激进的调度程序由于忽略输出内存要求，导致频繁驱逐请求，导致吞吐量先增加然后下降。相比之下，我们提出的调度程序可确保在各种 LLM 场景中保持稳定的性能，无论输入输出长度分布如何。

5.3 消融研究

在这一部分中，我们从内存利用率和请求驱逐两个角度对 decode-heavy Distribution-1、balanced Distribution-2 和 prefill-heavy Distribution-3 上的 Past-Future 调度器进行深入分析。表 1

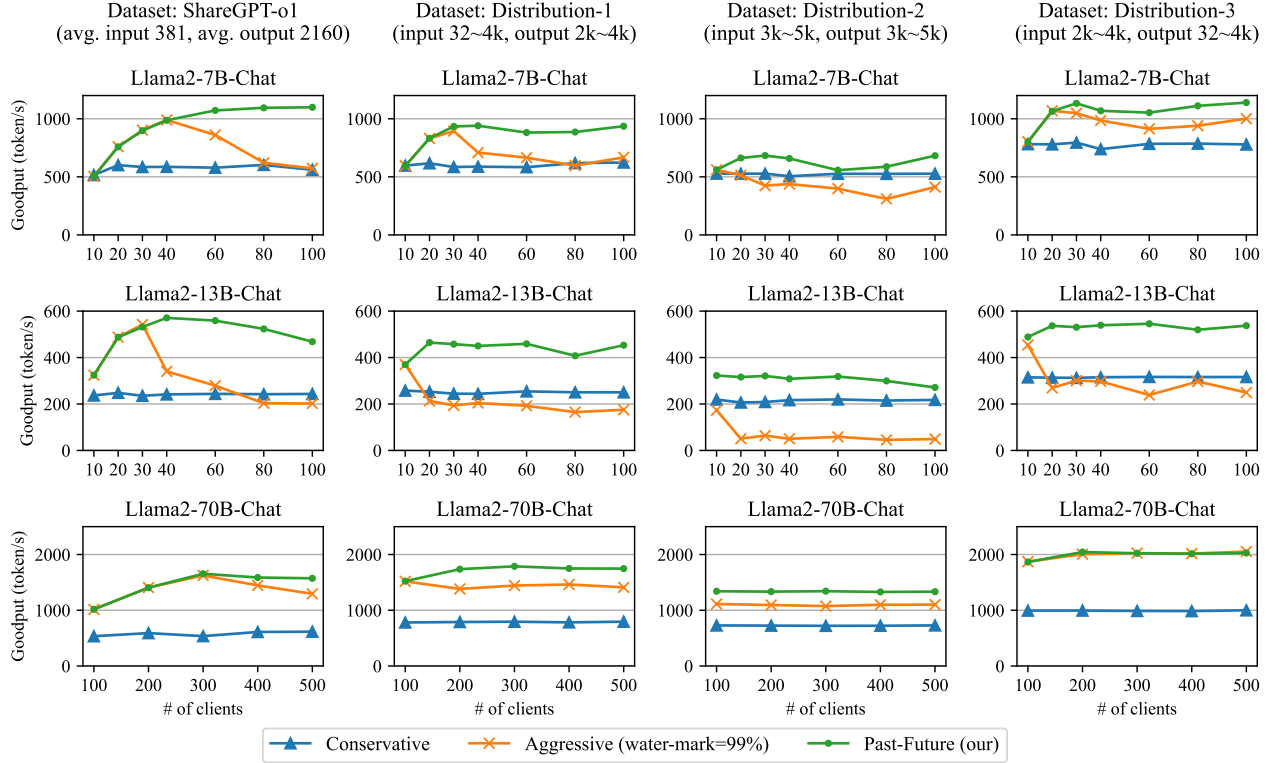


Figure 7. Detailed goodput under different numbers of clients for different schedulers. Conservative schedulers can only achieve low goodput at all times, while aggressive schedulers, although they can achieve equally goodput under light loads, have a rapid degradation of goodput under high loads. Our Past-Future scheduler maintains a consistently high goodput.

lists the decoding steps, average memory utilization, average future required memory, and the percentage of evicted requests across different schedulers of Llama-2 7B Chat on NVIDIA A100 80G. We also added the overcommit support for Conservative (e.g. overcommit=150% means that the scheduler assumes 1.5 times the actual memory capacity, we reduce the overcommit ratio for the Conservative scheduler in Distribution-2 due to too many evictions).

In Table 1, the Decoding Steps represent the number of decodes required by continuous batching to perform these requests, a higher value means that these requests are being performed with a smaller batch size per decode and more times of decoding, which negatively affects the throughput performance. Current Consumed Memory is the average memory utilization during runtime, and the Future Required Memory is the actual memory requirement in the future to finish all current added requests when scheduling, more than 100% means that the new requests added at this time will lead to a lack of memory in the future, the table shows the average value. Evicted Reqs is the ratio of the number of request evictions to the total number of requests. Request evictions not only violate the SLA of the requests, but also lead to a reduction in throughput performance due to the need for recomputation or swapping.

We also performed a detailed evaluation of various scheduler parameter configurations. We constructed the test dataset by concatenating the ShareGPT-01 dataset followed by the Distribution-1, Distribution-2, and Distribution-3 in sequence to generate a workload with varying output length distributions. The results shown in Figure 8 demonstrate that the Past-Future scheduler outperforms both the aggressive and conservative schedulers under all configurations.

Effect on Memory Utilization. It can be seen that the conservative scheduler, assuming that each request is expected to generate *max_new_token* outputs, results in the lowest memory utilization and takes the most decoding steps. The aggressive scheduler, which does not consider the memory occupancy of request outputs, processes as many requests as possible simultaneously until achieving a given memory watermark, it can achieve high memory utilization. In contrast, the proposed Past-Future scheduler which estimates the memory occupancy of each request at each future time step based on the dynamically predicted output lengths sampled from the probability distribution of historical requests, can achieve high GPU memory utilization close to the theoretical optimum on all datasets. The theoretical optimum represents the optimal case where the memory is optimally

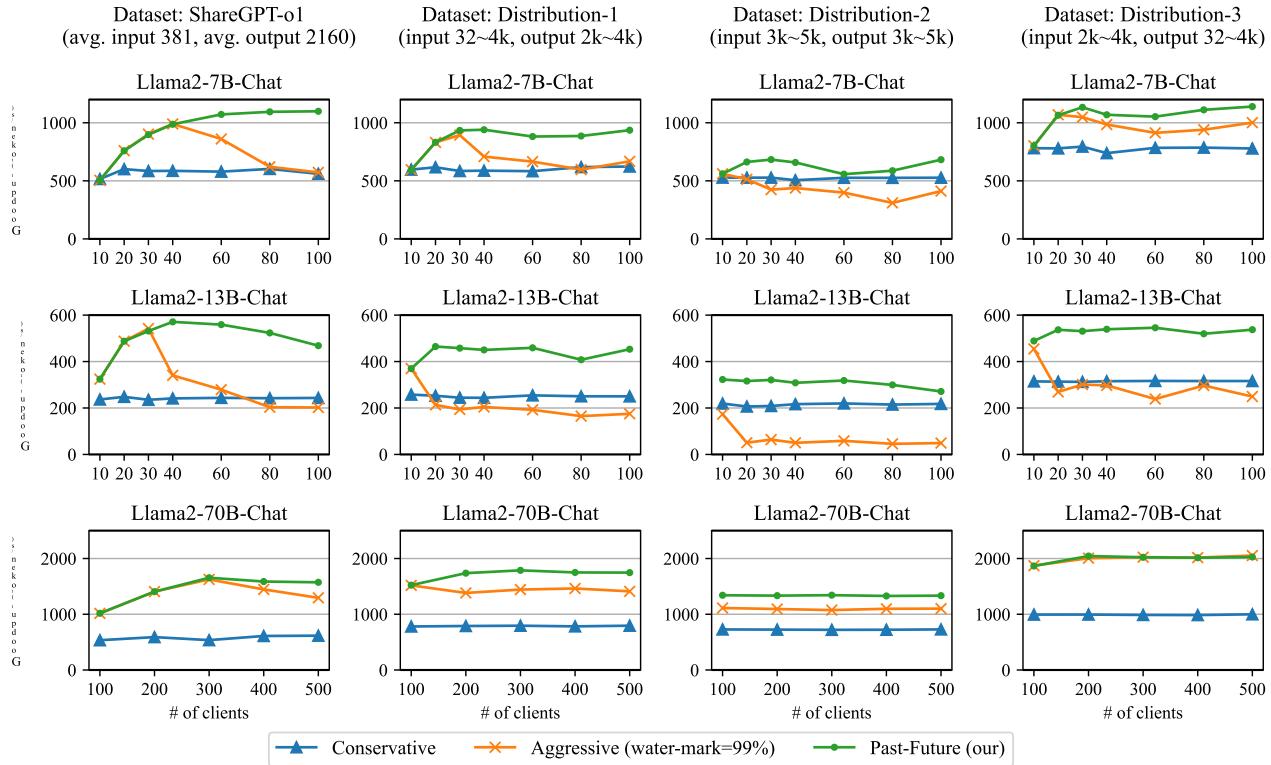


图 7. 不同调度程序在不同客户端数量下的详细吞吐量。保守的调度程序在任何时候都只能实现较低的吞吐量，而激进的调度程序虽然可以在轻负载下实现相同的吞吐量，但在高负载下吞吐量会快速下降。我们的“过去-未来”调度程序保持一贯的高吞吐量。

列出了 NVIDIA A100 80G 上 Llama-2 7B Chat 的不同调度程序的解码步骤、平均内存利用率、平均未来所需内存以及逐出请求的百分比。我们还添加了对 Conservative 的过度使用支持（例如，过度使用=150% 意味着调度程序假定实际内存容量的 1.5 倍，由于驱逐次数过多，我们降低了 Distribution-2 中 Conservative 调度程序的过度使用比率）。

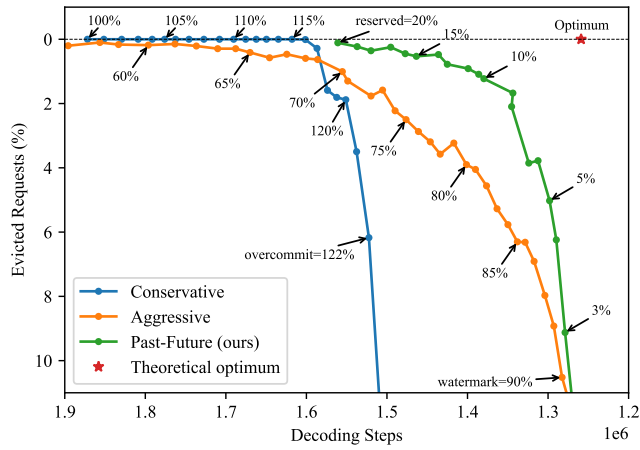
在表1中，解码步骤表示连续批处理执行这些请求所需的解码次数，较高的值意味着这些请求以每次解码的较小批量大小和更多解码次数执行，这会对吞吐量性能产生负面影响。Current Consumed Memory是运行时的平均内存利用率，FutureRequiredMemory是未来调度时完成当前添加的所有请求所需的实际内存需求，超过100%意味着此时添加的新请求将导致未来内存不足，表中显示的是平均值。Evicted Reqs 是请求驱逐数与请求总数的比率。请求驱逐不仅违反了请求的 SLA，而且还会由于需要重新计算或交换而导致吞吐量性能下降。

我们还对各种调度程序参数配置进行了详细评估。我们通过依次连接 ShareGPT-01 数据集和 Distribution-1、Distribution-2 和 Distribution-3 来构建测试数据集，以生成具有不同输出长度分布的工作负载。图 8 中显示的结果表明，过去未来调度程序在所有配置下均优于激进调度程序和保守调度程序。

对内存利用率的影响。可以看出，保守调度器假设每个请求都期望生成 `max_new_token` 输出，导致内存利用率最低并采取最多的解码步骤。主动调度器不考虑请求输出的内存占用，同时处理尽可能多的请求，直到达到给定的内存水印，它可以实现较高的内存利用率。相比之下，所提出的过去未来调度器根据从历史请求的概率分布中采样的动态预测的输出长度来估计每个未来时间步的每个请求的内存占用率，可以在所有数据集上实现接近理论最佳值的高GPU内存利用率。理论最优表示内存最优的情况

Table 1. Performance of different scheduling methods on multiple datasets.

Dataset	Method	Decoding Steps	Current Consumed Memory	Future Required Memory	Evicted Reqs
Distribution-1 (Decode-heavy)	Theoretical optimum	294250	94.87%	98.89%	0%
	Past-Future (reserved=3%)	296710	93.61%	97.47%	6.86%
	Past-Future (reserved=5%)	301680	91.87%	95.73%	3.37%
	Past-Future (reserved=10%)	320720	87.09%	91.51%	1.58%
	Aggressive (watermark=99%)	284160	98.41%	103.12%	93.74%
	Aggressive (watermark=95%)	290980	96.10%	101.21%	24.08%
	Aggressive (watermark=90%)	302900	92.46%	97.87%	10.76%
	Conservative (no overcommit)	485650	57.75%	60.79%	0%
	Conservative (overcommit=150%)	320530	87.34%	90.79%	17.23%
Distribution-2 (Balanced)	Theoretical optimum	653120	92.57%	98.29%	0%
	Past-Future (reserved=3%)	654500	92.16%	97.21%	7.42%
	Past-Future (reserved=5%)	669770	90.07%	95.82%	4.39%
	Past-Future (reserved=10%)	705920	85.26%	90.99%	1.54%
	Aggressive (watermark=99%)	616230	97.83%	103.68%	97.72%
	Aggressive (watermark=95%)	628410	95.86%	101.81%	26.19%
	Aggressive (watermark=90%)	654190	92.37%	99.12%	13.88%
	Conservative (no overcommit)	842470	71.71%	75.45%	0%
	Conservative (overcommit=125%)	665970	90.64%	94.68%	84.34%
Distribution-3 (Prefill-heavy)	Theoretical optimum	230690	96.60%	98.60%	0%
	Past-Future (reserved=3%)	239260	94.50%	96.51%	2.59%
	Past-Future (reserved=5%)	241650	92.64%	94.62%	0.87%
	Past-Future (reserved=10%)	263670	87.80%	89.87%	0.00%
	Aggressive (watermark=99%)	227250	98.13%	100.20%	34.30%
	Aggressive (watermark=95%)	234540	95.07%	97.16%	3.23%
	Aggressive (watermark=90%)	247130	90.29%	92.33%	0.17%
	Conservative (no overcommit)	374250	59.73%	61.59%	0%
	Conservative (overcommit=150%)	246870	90.45%	92.54%	19.09%

**Figure 8.** Comparison of different scheduler parameters on a varying load consisting of ShareGPT-o1 and Distribution-1 to 3.

utilized when the request output length is known, which is impossible in real-world service.

Effect on Request Eviction. Although an aggressive scheduler with a high memory watermark or a conservative scheduler with overcommit can also achieve high memory utilization, it introduces a significant amount of request eviction as shown in Table 1 (93.74% on decode-heavy *Distribution-1* for aggressive scheduler with watermark=99%). Tuning the watermark or overcommit ratio helps to reduce evicted requests, but decoding steps also increase. At the same time, this tuning is more difficult to adapt to the workload with variable distribution, we further analyze the different parameters of different schedulers in this case, as shown in Figure 8, both aggressive and conservative have difficulty in balancing the evicted requests and decoding steps, and cannot achieve the effect of Past-Future scheduler by parameter tuning. Since aggressive and conservative schedulers are not able to sense the actual load, their evicted requests increase rapidly when approaching the memory capacity threshold. The superior performance of the Future-Past scheduler arises from its ability to sense the workload through historical distribution, enabling it to accurately predict future memory requirements. Although it incorporates a percentage of reserved memory in its scheduling decisions to handle minor fluctuations in

表 1. 不同调度方法在多个数据集上的性能。

Dataset	Method	Decoding Steps	Current Consumed Memory	Future Required Memory	Evicted Reqs
Distribution-1 (Decode-heavy)	Theoretical optimum	294250	94.87%	98.89%	0%
	Past-Future (reserved=3%)	296710	93.61%	97.47%	6.86%
	Past-Future (reserved=5%)	301680	91.87%	95.73%	3.37%
	Past-Future (reserved=10%)	320720	87.09%	91.51%	1.58%
	Aggressive (watermark=99%)	284160	98.41%	103.12%	93.74%
	Aggressive (watermark=95%)	290980	96.10%	101.21%	24.08%
	Aggressive (watermark=90%)	302900	92.46%	97.87%	10.76%
	Conservative (no overcommit)	485650	57.75%	60.79%	0%
	Conservative (overcommit=150%)	320530	87.34%	90.79%	17.23%
Distribution-2 (Balanced)	Theoretical optimum	653120	92.57%	98.29%	0%
	Past-Future (reserved=3%)	654500	92.16%	97.21%	7.42%
	Past-Future (reserved=5%)	669770	90.07%	95.82%	4.39%
	Past-Future (reserved=10%)	705920	85.26%	90.99%	1.54%
	Aggressive (watermark=99%)	616230	97.83%	103.68%	97.72%
	Aggressive (watermark=95%)	628410	95.86%	101.81%	26.19%
	Aggressive (watermark=90%)	654190	92.37%	99.12%	13.88%
	Conservative (no overcommit)	842470	71.71%	75.45%	0%
	Conservative (overcommit=125%)	665970	90.64%	94.68%	84.34%
Distribution-3 (Prefill-heavy)	Theoretical optimum	230690	96.60%	98.60%	0%
	Past-Future (reserved=3%)	239260	94.50%	96.51%	2.59%
	Past-Future (reserved=5%)	241650	92.64%	94.62%	0.87%
	Past-Future (reserved=10%)	263670	87.80%	89.87%	0.00%
	Aggressive (watermark=99%)	227250	98.13%	100.20%	34.30%
	Aggressive (watermark=95%)	234540	95.07%	97.16%	3.23%
	Aggressive (watermark=90%)	247130	90.29%	92.33%	0.17%
	Conservative (no overcommit)	374250	59.73%	61.59%	0%
	Conservative (overcommit=150%)	246870	90.45%	92.54%	19.09%

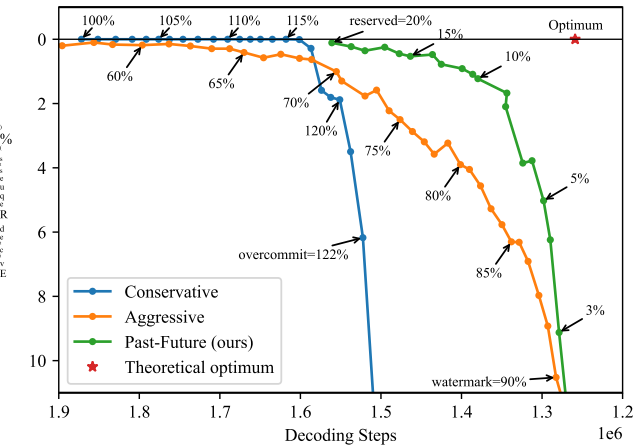


图 8. 由 ShareGPT-o1 和 Distribution-1 到 3 组成的不同负载上不同调度程序参数的比较。

当请求输出长度已知时使用，这在实际服务中是不可能的。

对请求驱逐的影响。虽然具有高内存水印的激进调度器或具有过度使用的保守调度器也可以实现高内存利用率，但它引入了大量的请求驱逐，如表1所示（对于具有水印的激进调度器=99%，在大量解码的Distribution-1上为93.74%）。调整水印或过度使用比率有助于减少被逐出的请求，但解码步骤也会增加。同时，这种调优更难以适应可变分布的工作负载，我们进一步分析了这种情况下不同调度器的不同参数，如图8所示，激进式和保守式都难以平衡驱逐请求和解码步骤，并且无法通过参数调优达到Past-Future调度器的效果。由于激进型和保守型调度程序无法感知实际负载，因此当接近内存容量阈值时，它们的逐出请求会迅速增加。未来-过去调度程序的卓越性能源于其通过历史分布感知工作负载的能力，使其能够准确预测未来的内存需求。尽管它在调度决策中纳入了一定比例的保留内存来处理较小的波动

request distribution, it consistently outperforms other schedulers across all reserved ratio configurations, and the eviction rate exhibits smoother variation within an appropriate range.

5.4 End-to-end Performance

To demonstrate that our improvements were not achieved based on a lower baseline framework, we compare LightLLM with the proposed Past-Future scheduler with the currently popular LLM service frameworks, including Text-Generation-Inference (TGI) [12] with conservative scheduler, vLLM [13] with aggressive scheduler, and DeepSpeed-MII (FastGen) [17] with conservative scheduler and the splitfuse strategy of the recent version on the commonly-used ShareGPT dataset. To simulate the unknown output length in a real-world scenario, we set a maximum output length $max_new_tokens = 2048$ for each request. Requests will be completed when they reach the maximum length or encounter an end token. We also implemented a conservative scheduler for the TensorRT-LLM [20] backend.

We evaluate the maximum throughput and the goodput under the SLA guarantee of each framework which adopts different scheduling strategies by simulating concurrent requests from different numbers of clients. Figure 9 presents the performance across different hardware platforms and model sizes. It can be observed that with the help of the proposed Past-Future scheduler, LightLLM performed a competitive performance on maximum throughput, and a superior goodput in all scenarios. Additionally, we find that TGI and DeepSpeed-MII, employing a conservative scheduling approach, fail to achieve high maximum throughput, suggesting that a conservative scheduling strategy can lead to the wastage of computational resources and memory resources. Even with the fast static inference speed of TensorRT-LLM, it is difficult to achieve satisfactory throughput with a conservative scheduler.

In contrast, vLLM, which adopts an aggressive scheduler, achieves better maximum throughput performance with 7B and 13B models. However, due to the aggressive scheduler, which can more easily introduce harmful request eviction and re-queue, there is a noticeable decline in goodput under the SLA guarantee. Equipped with the Past-Future Scheduler, LightLLM can maximize its maximum throughput while avoiding harmful request eviction, thus ensuring goodput under the SLA guarantee.¹

5.5 Performance on Multimodal LLM

We also validate the compatibility of the proposed Past-Future scheduler on multimodal tasks. Table 2 presents the

¹The performance data for each framework, including LightLLM, in Figure 9 is based on the version from December 2023. The comparison across frameworks is only intended to show that our scheduler achieves improvements based on a strong baseline. Each framework has seen significant progress and development since then, and we will release the updated results in our open-source code repository.

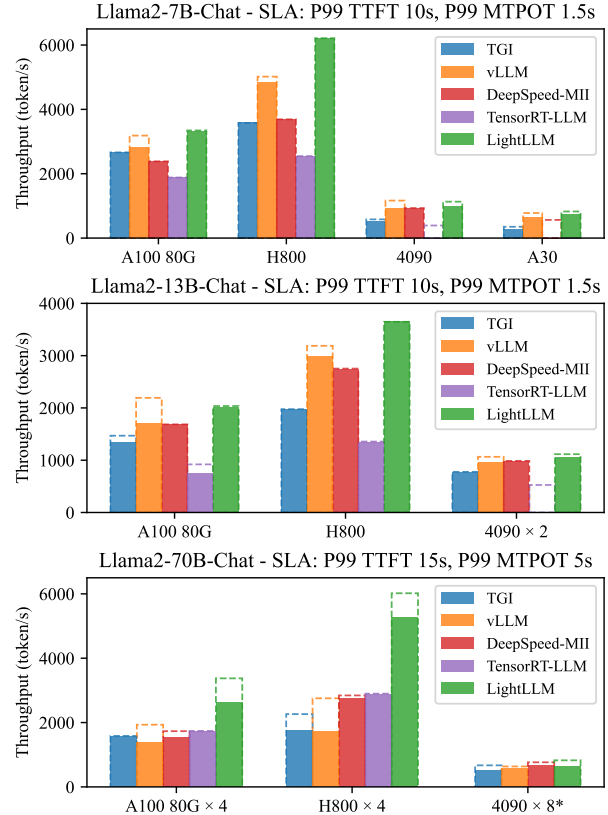


Figure 9. Throughput (dashed) and SLA-guarantee goodput (solid) of different frameworks across various hardware. SLA constraints marked as *P99 TTFT 10s, P99 MTPOT 1.5s* refers to the 99% of the requests’ first token latency must be $< 10s$, and the requests’ maximum per-token latency for 99% requests must be $< 1.5s$.

throughput comparison between the original implementations and LightLLM with the Past-Future scheduler for Qwen-VL-Chat, LLaVA-1.5-7B, and LLaVA-1.5-13B. Due to the Past-Future scheduler’s ability to effectively improve system memory utilization, we achieved a 50% throughput improvement on Qwen-VL-Chat and a 60% improvement on LLaVA-1.5-7B compared to their original implementation.

Table 2. Performance comparison of LightLLM on the Qwen-VL-Chat, LLaVA-1.5-7B and LLaVA-1.5-13B with their original implementation.

Model	Throughput (tokens/s)	
	Origin	LightLLM
Qwen-VL-Chat	219.96	319.19
Llava-1.5-7B	535.31	851.73
Llava-1.5-13B	1193.38	2228.71

在请求分配方面，它在所有保留比率配置上始终优于其他调度程序，并且逐出率在适当的范围内表现出更平滑的变化。

5.4 端到端性能

为了证明我们的改进不是基于较低基线框架实现的，我们将 LightLLM 与提出的过去未来调度器与当前流行的 LLM 服务框架进行比较，包括带有保守调度器的文本生成推理 (TGI) [12]、带有激进调度器的 vLLM [13]、带有保守调度器的 DeepSpeed-MII (FastGen) [17] 以及常用 ShareGPT 数据集上最新版本的 splitfuse 策略。为了模拟现实场景中的未知输出长度，我们为每个请求设置最大输出长度 $max_new_tokens = 2048$ 。当请求达到最大长度或遇到结束令牌时，请求将完成。我们还为 TensorRT-LLM [20] 后端实现了保守的调度程序。

我们通过模拟来自不同数量客户端的并发请求来评估采用不同调度策略的每个框架在SLA保证下的最大吞吐量和吞吐量。图 9 显示了不同硬件平台和模型大小的性能。可以看出，在所提出的过去未来调度程序的帮助下，LightLLM 在最大吞吐量方面表现出有竞争力的性能，并且在所有场景中都具有出色的吞吐量。此外，我们发现TGI和DeepSpeed-MII采用保守的调度方法，无法实现高的最大吞吐量，这表明保守的调度策略会导致计算资源和内存资源的浪费。即使 TensorRT-LLM 具有快速的静态推理速度，也很难通过保守的调度器获得令人满意的吞吐量。

相比之下，采用积极调度器的vLLM在7B和13B模型中实现了更好的最大吞吐量性能。然而，由于激进的调度程序更容易引入有害的请求驱逐和重新排队，因此SLA保证下的吞吐量显著下降。LightLLM配备了Past-Future Scheduler，可以最大化其最大吞吐量，同时避免有害的请求驱逐，从而确保SLA保证下的良好吞吐量。¹

5.5 多模式大语言模型

我们还验证了所提出的过去-未来调度程序在多模式任务上的兼容性。表 2 列出了

¹The performance data for each framework, including LightLLM, in Figure 9 is based on the version from December 2023. The comparison across frameworks is only intended to show that our scheduler achieves improvements based on a strong baseline. Each framework has seen significant progress and development since then, and we will release the updated results in our open-source code repository.

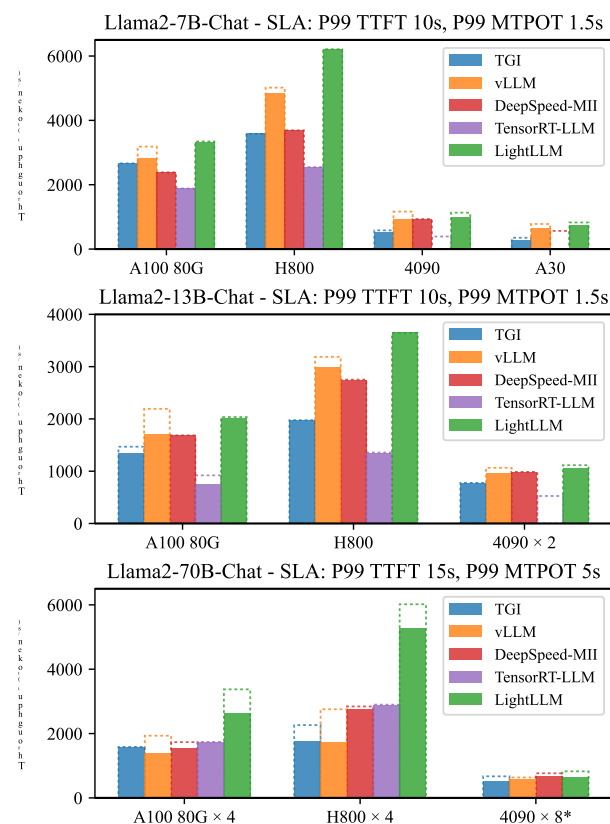


图 9. 跨各种硬件的不同框架的吞吐量（虚线）和 SLA 保证良好吞吐量（实线）。SLA 约束标记为 P99 TTFT 10s、P99 MTPOT 1.5s 是指 99% 的请求的首次令牌延迟必须为 $< 10s$ ，并且 99% 的请求的最大每个令牌延迟必须为 $< 1.5s$ 。

Qwen-VL-Chat、LLaVA-1.5-7B 和 LLaVA-1.5-13B 的原始实现与带有过去未来调度程序的 LightLLM 之间的吞吐量比较。由于过去-未来调度程序能够有效提高系统内存利用率，与原始实现相比，我们在 Qwen-VL-Chat 上实现了 50% 的吞吐量改进，在 LLaVA-1.5-7B 上实现了 60% 的吞吐量改进。

表 2. Qwen-VL-Chat、LLaVA-1.5-7B 和 LLaVA-1.5-13B 上的 LightLLM 与其原始实现的性能比较。

Model	Throughput (tokens/s)	
	Origin	LightLLM
Qwen-VL-Chat	219.96	319.19
Llava-1.5-7B	535.31	851.73
Llava-1.5-13B	1193.38	2228.71

6 Related Work

With the popularity of large-scale model applications, there is an increasing demand for deploying high-performance large models. Currently, there are some excellent frameworks for serving large models. They optimize the throughput performance of large model services from multiple perspectives, including parallelization, GPU operator optimization, request batching strategies, and GPU memory management.

Parallelization is the most commonly used technique to accelerate the LLM inference which includes data parallelism, pipeline parallelism, tensor parallelism, and sequence parallelism. Data parallelism and sequence parallelism are often employed for accelerating LLM training. Pipeline parallelism and tensor parallelism are widely applied to large model inference. Pipeline parallelism splits the model into multiple GPUs, with each GPU loading one or more layers of the LLM. While tensor parallelism involves splitting the model’s parameters (tensors) across different devices (such as GPUs) and performing computations in parallel. Both of them can make the system handle the substantial parameter sizes of these models and achieve superior overall performance.

GPU operator optimization is also an effective way to speed up the inference because there are many memory-bound operations in LLM inference, e.g., layernorm and attention mechanisms. TensorRT-LLM, DeepSpeed, and similar frameworks leverage CUDA customization to optimize these operations, resulting in speed improvements. Tri Dao et al. [6] introduce Flash Attention, which performs attention calculations on SRAM with higher bandwidth to reduce memory access overhead. Building upon this, they further designed Flash AttentionV2 [5], Flash Decoding, and Flash Decoding++ [11].

Batching requests can effectively enhance the throughput of LLM systems. The most commonly used method is iteration-level scheduling, also known as continuous batching, introduced by ORCA [40]. In this approach, during the autoregressive decoding process, at each step, the system dynamically assesses whether requests have been completed and whether new requests can be added, improving the overall throughput.

Memory management is also a critical bottleneck affecting the throughput of LLM systems. This is because the autoregressive encoding process in LLM generates a significant amount of KV cache, leading to the issue of memory fragmentation. Techniques such as recomputation and dynamic offloading can reduce peak memory usage, thereby increasing the system’s request capacity. However, they come with additional computational or memory access overhead. PageAttention, introduced by vLLM, manages the KV cache in blocks, inspired by the paging techniques in operating systems. It effectively reduces memory fragmentation issues but still inevitably involves some memory wastage.

7 Conclusion and Future Work

In this paper, we focus on optimizing goodput under SLA guarantees and introduce a novel Past-Future scheduler that consists of looking to the future and reflecting on the past. It not only considers memory usage at each future time step but also leverages the historical request output length distribution to dynamically predict the output length of requests in the running batch, maximizing memory utilization and minimizing harmful request evictions. Additionally, we present LightLLM, a high-performance LLM serving framework. With the proposed Past-Future Scheduler, LightLLM can be deployed on a cluster with thousands of GPUs to support a variety of real-world LLM services.

For future work, we believe the proposed scheduler also has significant potential for the scenarios of dynamic service instance availability and fluctuating workloads. Based on its ability to accurately estimate the memory demand of each running batch, it can forward requests to under-utilized services based on the current SLA constraints and service load, aiming to ensure that each service reaches full capacity. Therefore, when the request workload changes, it can achieve timely machine scaling with minimal request migration overhead.

Acknowledgements

We thank all reviewers for their insightful comments. This work is supported by the Beijing Municipal Science and Technology Project (No. Z231100010323002), the National Natural Science Foundation of China (No. 62322201, U23B2020, 62306025, 92367204), CCF-Baidu Open Fund and the Fundamental Research Funds for the Central Universities (JK2024-58).

References

- [1] Anthropic. 2023. Anthropic Claude. <https://claude.ai/>
- [2] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, et al. 2023. Qwen Technical Report. *arXiv preprint arXiv:2309.16609* (2023).
- [3] Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, Sinan Tan, Peng Wang, Junyang Lin, Chang Zhou, and Jingren Zhou. 2023. Qwen-VL: A Frontier Large Vision-Language Model with Versatile Abilities. *arXiv preprint arXiv:2308.12966* (2023).
- [4] Baichuan. 2023. Baichuan 2: Open Large-scale Language Models. *arXiv preprint arXiv:2309.10305* (2023). <https://arxiv.org/abs/2309.10305>
- [5] Tri Dao. 2023. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. (2023).
- [6] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems*.
- [7] Zhengxiao Du, Yujie Qian, Xiao Liu, Ming Ding, Jiezhong Qiu, Zhilin Yang, and Jie Tang. 2022. GLM: General Language Model Pretraining with Autoregressive Blank Infilling. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 320–335.
- [8] Elias Frantar, Saleh Ashkboos, Torsten Hoeffler, and Dan Alistarh. 2022. GPTQ: Accurate Post-training Compression for Generative Pretrained Transformers. *arXiv preprint arXiv:2210.17323* (2022).
- [9] Github. 2022. Github Copilot. <https://github.com/features/copilot>

6 相关工作

随着大规模模型应用的普及，部署高性能大型模型的需求越来越大。目前，有一些优秀的框架可以服务于大型模型。他们从多个角度优化大型模型服务的吞吐量性能，包括并行化、GPU算子优化、请求批处理策略和GPU内存管理。

并行化是加速LLM推理最常用的技术，包括数据并行、管道并行、张量并行和序列并行。数据并行和序列并行通常用于加速 LLM 训练。管道并行和张量并行广泛应用于大型模型推理。管道并行性将模型拆分为多个 GPU，每个 GPU 加载一层或多层 LLM。张量并行涉及将模型的参数（张量）拆分到不同的设备（例如 GPU）上并并行执行计算。它们都可以使系统处理这些模型的大量参数大小并获得优越的整体性能。

GPU算子优化也是加速推理的有效方法，因为LLM推理中有许多内存限制操作，例如层范数和注意力机制。TensorRT-LLM、DeepSpeed 和类似框架利用 CUDA 定制来优化这些操作，从而提高速度。Tri Dao 等人。[6] 引入了Flash Attention，它在具有更高带宽的SRAM上执行注意力计算，以减少内存访问开销。在此基础上，他们进一步设计了 Flash AttentionV2 [5]、Flash Decoding 和 Flash Decoding++ [11]。

批量请求可以有效提高LLM系统的吞吐量。最常用的方法是迭代级调度，也称为连续批处理，由 ORCA [40] 引入。在这种方法中，在自回归解码过程中，系统在每一步动态评估请求是否已完成以及是否可以添加新请求，从而提高整体吞吐量。

内存管理也是影响 LLM 系统吞吐量的关键瓶颈。这是因为LLM中的自回归编码过程会生成大量的KV缓存，从而导致内存碎片问题。重新计算和动态卸载等技术可以减少峰值内存使用量，从而提高系统的请求容量。然而，它们会带来额外的计算或内存访问开销。PageAttention 由 vLLM 引入，受操作系统中分页技术的启发，以块的形式管理 KV 缓存。它有效地减少了内存碎片问题，但仍然不可避免地涉及一些内存浪费。

7 结论和未来工作

在本文中，我们重点关注在 SLA 保证下优化有效吞吐量，并介绍一种新颖的过去未来调度程序，该调度程序包括展望未来和反思过去。它不仅考虑未来每个时间步的内存使用情况，还利用历史请求输出长度分布来动态预测正在运行的批处理中请求的输出长度，从而最大限度地提高内存利用率并最大限度地减少有害的请求驱逐。此外，我们还推出了 LightLLM，一个高性能的 LLM 服务框架。通过提出的过去未来调度程序，LightLLM 可以部署在具有数千个 GPU 的集群上，以支持各种现实世界的 LLM 服务。

对于未来的工作，我们相信所提出的调度程序对于动态服务实例可用性和波动工作负载的场景也具有巨大的潜力。基于其准确估计每个运行批次的内存需求的能力，它可以根据当前的 SLA 约束和服务负载将请求转发到未充分利用的服务，旨在确保每个服务达到满负荷。因此，当请求工作负载发生变化时，能够以最小的请求迁移开销实现及时的机器伸缩。

致谢

我们感谢所有审稿人富有洞察力的评论。该工作得到了北京市科技计划项目（No. Z231100010323002）、国家自然科学基金项目（No. 62322201、U23B2020、62306025、92367204）、CCF-百度开放基金和中央高校基本科研业务费专项资金的资助（JK2024-58）。

参考

- [1] 人择。2023.人类克劳德。 <https://claude.ai/> [2] 白金泽，白帅，褚云飞，崔泽宇，党凯，等。2023.Qwen 技术报告。arXiv 预印本 arXiv:2309.16609 (2023)。 [3] 白金泽，白帅，杨树生，王士杰，谭思南，王鹏，林俊阳，周常，周景仁。2023. Qwen-VL: 具有多功能能力的前沿大型视觉语言模型。arXiv 预印本 arXiv:2308.12966 (2023)。 [4] 百川。2023.百川2: 开放大规模语言模型。arXiv 预印本 arXiv: 2309.10305 (2023)。 <https://arxiv.org/abs/2309.10305> [5] 三道。2023. FlashAttention-2: 更快的注意力，更好的并行性和工作分区。（2023）。 [6] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra 和 Christopher Ré. 2022. FlashAttention: 具有 IO 感知的快速、内存高效的精确注意力。神经信息处理系统的进展。 [7] 杜正晓，钱玉杰，刘晓，丁明，邱杰中，杨志林，唐杰。2022.GLM: 具有自回归空白填充的通用语言模型预训练。计算语言学协会第 60 届年会论文集（第一卷：长论文）。320–335。 [8] Elias Frantar, Saleh Ashkboos, Torsten Hoefler 和 Dan Alistarh. 2022.GPTQ: 生成预训练 Transformer 的精确训练后压缩。arXiv 预印本 arXiv:2210.17323 (2022)。 [9] 吉图布。2022.Github 副驾驶。 <https://github.com/features/copilot>

- [10] Google. 2023. Google Bard. <https://bard.google.com/>
- [11] Ke Hong, Guohao Dai, Jiaming Xu, Qiuli Mao, Xiuhong Li, Jun Liu, Kangdi Chen, Hanyu Dong, and Yu Wang. 2023. FlashDecoding++: Faster Large Language Model Inference on GPUs. *arXiv preprint arXiv:2311.01282* (2023).
- [12] HuggingFace. 2023. Text Generation Inference. <https://github.com/huggingface/text-generation-inference>.
- [13] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.
- [14] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, et al. 2023. StarCoder: may the source be with you! *arXiv:2305.06161* [cs.CL]
- [15] Haotian Liu, Chunyuan Li, Yuheng Li, and Yong Jae Lee. 2023. Improved Baselines with Visual Instruction Tuning. *arXiv:2310.03744* [cs.CV]
- [16] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2023. Visual Instruction Tuning. In *NeurIPS*.
- [17] Microsoft. 2023. DeepSpeed-FastGen: High-throughput Text Generation for LLMs via MII and DeepSpeed-Inference. <https://github.com/microsoft/DeepSpeed/tree/master/blogs/deepspeed-fastgen>.
- [18] Ramesh Nallapati, Bowen Zhou, Cicero dos Santos, Caglar Gulcehre, and Bing Xiang. 2016. Abstractive Text Summarization using Sequence-to-sequence RNNs and Beyond. Association for Computational Linguistics, Berlin, Germany, 280–290. <https://doi.org/10.18653/v1/K16-1028>
- [19] NVIDIA. 2021. FasterTransformer. <https://github.com/NVIDIA/FasterTransformer/>
- [20] NVIDIA. 2023. TensorRT-LLM. <https://github.com/NVIDIA/TensorRT-LLM/>.
- [21] OpenAI. 2022. OpenAI ChatGPT. <https://openai.com/blog/chatgpt>
- [22] OpenAI. 2023. GPT-4 Technical Report. *arXiv:2303.08774* [cs.CL]
- [23] Adam Paszke, Sam Gross, Soumith Chintala, Gregory Chanan, Edward Yang, Zachary DeVito, Zeming Lin, Alban Desmaison, Luca Antiga, and Adam Lerer. 2017. Automatic differentiation in PyTorch. (2017).
- [24] Romain Paulus, Caiming Xiong, and Richard Socher. 2018. A Deep Reinforced Model for Abstractive Summarization. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=HkACIQgA->
- [25] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, , and Xinran Xu. 2024. Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving. (2024). <https://arxiv.org/abs/2407.00079>
- [26] Abigail See, Peter J. Liu, and Christopher D. Manning. 2017. Get To The Point: Summarization with Pointer-Generator Networks. Association for Computational Linguistics, Vancouver, Canada, 1073–1083. <https://doi.org/10.18653/v1/P17-1099>
- [27] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Re, Ion Stoica, and Ce Zhang. 2023. FlexGen: high-throughput generative inference of large language models with a single GPU. In *International Conference on Machine Learning*. PMLR, 31094–31116.
- [28] Amanpreet Singh, Vivek Natarajan, Meet Shah, Yu Jiang, Xinlei Chen, Devi Parikh, and Marcus Rohrbach. 2019. Towards VQA Models That Can Read. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 8317–8326.
- [29] 01-AI Team. 2023. Yi LLM. <https://github.com/01-ai/Yi>.
- [30] InternLM Team. 2023. InternLM: A Multilingual Language Model with Progressively Enhanced Capabilities. <https://github.com/InternLM/InternLM-techreport>.
- [31] Philippe Tillet, H. T. Kung, and David Cox. 2019. Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages* (Phoenix, AZ, USA) (MAPL 2019). Association for Computing Machinery, New York, NY, USA, 10–19. <https://doi.org/10.1145/3315508.3329973>
- [32] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [33] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, et al. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv:2307.09288* [cs.CL]
- [34] Yuxin Wang, Yuhang Chen, Zeyu Li, Zhenheng Tang, Rui Guo, Xin Wang, Qiang Wang, Amelie Chi Zhou, and Xiaowen Chu. 2024. Towards Efficient and Reliable LLM Serving: A Real-World Workload Study. *arXiv:2401.17644* [cs.DC]
- [35] Jason Wei, Yi Tay, Rishi Bommasani, Barret Zoph Colin Raffel, Sebastian Borgeaud, Dani Yogatama, et al. 2022. Emergent Abilities of Large Language Models. *Transactions on Machine Learning Research* (2022). <https://openreview.net/forum?id=yzkSU5zdwD> Survey Certification.
- [36] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, brian ichter, Fei Xia, Ed H. Chi, Quoc V Le, and Denny Zhou. 2022. Chain of Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems*, Alice H. Oh, Alekh Agarwal, Danielle Belgrave, and Kyunghyun Cho (Eds.). https://openreview.net/forum?id=_VjQMeSB_J
- [37] Xiuying Wei, Yunchen Zhang, Yuhang Li, Xiangguo Zhang, Ruihao Gong, Jinyang Guo, and Xianglong Liu. 2023. Outlier Suppression+: Accurate quantization of large language models by equivalent and effective shifting and scaling. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Singapore, 1648–1665.
- [38] Wikipedia. 2023. Service-level agreement. (2023). https://en.wikipedia.org/wiki/Service-level_agreement [Online; accessed 24-October-2023].
- [39] BigScience Workshop, Teven Le Scao, Angela Fan, Christopher Akiki, et al. 2023. BLOOM: A 176B-Parameter Open-Access Multilingual Language Model. *arXiv:2211.05100* [cs.CL]
- [40] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 521–538.
- [41] Aohan Zeng, Xiao Liu, Zhengxiao Du, Zihan Wang, Hanyu Lai, Ming Ding, Zhuoyi Yang, Yifan Xu, Wendi Zheng, Xiao Xia, et al. 2022. Glm-130b: An open bilingual pre-trained model. *arXiv preprint arXiv:2210.02414* (2022).

- [10]谷歌。2023.谷歌吟游诗人。https://bard.google.com/ [11]洪科、戴国豪、徐家明、毛秋丽、李秀红、刘军、陈康迪、董韩宇和王宇。2023.FlashDecoding++: GPU 上更快的大型语言模型推理。arXiv 预印本 arXiv:2311.01282 (2023)。[12]拥抱脸。2023.文本生成推理。https://github.com/huggingface/text-generation-inference。[13]Woosuk Kwon, Zhuohan Li, 庄思源, Yingheng, Lianmin Cheng, Codyhao Yu, Joseph Gonzalez, Hao Zhang, Ion Stoica。2023.通过 pagedattention 服务的大型语言模型的高效内存管理。第 29 届操作系统原理研讨会论文集。611–626。[14]Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, 等。2023.StarCoder: 愿源头与你同在! arXiv:2305.06161 [cs.CL] [15]Haotian Liu, Chunyuan Li, Yuheng Li 和 Yong Jae Lee。2023.通过视觉指令调整改进基线。arXiv:2310.03744 [cs.CV] [16]Haotian Liu, Chunyuan Li, Qingyang Wu 和 Yong Jae Lee。2023.视觉指令调整。在 NeurIPS 中。[17]微软。2023.DeepSpeed-FastGen: 通过 MII 和 DeepSpeed-Inference 为 LLM 生成高通量文本。https://github.com/microsoft/DeepSpeed/tree/master/blogs/deepspeed-fastgen。[18]Ramesh Nallapati, Bowen Zhou, Cicero dos Santos, Caglar Gulcehre, Bing Xing。2016.使用序列到序列 RNN 等进行抽象文本摘要。计算语言学协会, 柏林, 德国, 280-290。https://doi.org/10.18653/v1/K16-1028 [19]NVIDIA。2021.FasterTransformer。https://github.com/NVIDIA/FasterTransformer/ [20]NVIDIA。2023.TensorRT-LLM。https://github.com/NVIDIA/TensorRT-LLM/。[21]开放人工智能。2022.Open AI ChatGPT。https://openai.com/blog/chatgpt [22]OpenAI。2023.GPT-4 技术报告。arXiv:2303.08774 [cs.CL] [23]Adam Paszke, Sam Gross, Soumith Chintala, Gregory Chanan, Edward Yang, Zachary DeVito, Zeming Lin, Alban Desmaison, Luca Antiga 和 Adam Lerer。2017.PyTorch 中的自动微分。(2017)。[24]罗曼·保卢斯, 蔡明熊, 理查德·索彻。2018.抽象概括的深度强化模型。在国际学习表征会议上。https://openreview.net/forum?id=HkAClQgA- [25]秦若宇、李哲明、何蔚然、张明星、吴永伟、郑伟民、徐欣然。2024.Mooncake: 用于 LLM 服务的以 KVCache 为中心的分解架构。(2024)。https://arxiv.org/abs/2407.00079 [26]Abigail See, Peter J. Liu 和 Christopher D. Manning。2017.切入要点: 用指针生成器网络进行总结。计算语言学协会, 加拿大温哥华, 1073–1083。https://doi.org/10.18653/v1/P17-1099 [27]盛颖, 郑联民, 袁斌航, 李卓涵, Max Ryabinin, 陈贝迪, Percy Liang, Christopher Re, Ion Stoica 和张策。2023.FlexGen: 使用单个 GPU 对大型语言模型进行高吞吐量生成推理。在国际机器学习会议上。PMLR, 31094–31116。
- [28]Amanpreet Singh, Vivek Natarajan, Meet Shah, Yu Jiang, Xinlei Chen, Devi Parikh 和 Marcus Rohrbach。2019.迈向可读的 VQA 模型。IEEE 计算机视觉和模式识别会议论文集。8317–8326。[29]01-人工智能团队。2023.易大语言模型。https://github.com/01-ai/Yi。[30]实习生团队。2023.InternLM: 功能逐步增强的多语言语言模型。https://github.com/InternLM/InternLM-techreport。[31]Philippe Tillet, H. T. Kung 和 David Cox。2019.Triton: 用于平铺神经网络计算的中间语言和编译器。第三届 ACM SIGPLAN 国际机器学习和编程语言研讨会(美国亚利桑那州菲尼克斯)(MAPL 2019)论文集。计算机协会, 美国纽约州纽约市, 10-19。https://doi.org/10.1145/3315508.3329973 [32]Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar 等。2023.Llama: 开放高效的基础语言模型。arXiv 预印本 arXiv:2302.13971 (2023)。[33]Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi 等。2023.Llama 2: 开放基础和微调聊天模型。arXiv:2307.09288 [cs.CL] [34]Yuxin Wang, Yuhang Chen, Zeyu Li, Zhenheng Tang, Rui Guo, Xin Wang, Qianqiang Wang, Amelie Chi Zhou 和 Xiaowen Chu。2024.迈向高效可靠的大语言模型 现实世界工作量研究。arXiv:2401.17644 [cs.DC] [35]Jason Wei, Yi Tay, Rishi Bommasani, Barret Zoph Colin Raffel, Sebastian Borgeaud, Dani Yogatama 等。2022.大型语言模型的新兴能力。机器学习研究汇刊(2022)。https://openreview.net/forum?id=yzkSU5zdwd 调查认证。[36]Jason Wei, Xuezhi Wang, Dale Schuurmans, Maaarten Bosma, brian ichter, Fei Xia, Ed H. Chi, Quoc V Le 和 Denny Zhou。2022.思维链提示引发大型语言模型中的推理。《神经信息处理系统进展》, Alice H. Oh, Alekh Agarwal, Danielle Belgrave 和 Kyunghyun Cho (编辑)。https://openreview.net/forum?id=_VjQlMeSB_J [37]魏秀英, 张云晨, 李宇航, 张相国, 龚瑞浩, 郭金阳, 刘相龙。2023.异常值抑制+: 通过等效且有效的移位和缩放对大型语言模型进行精确量化。2023 年自然语言处理经验方法会议论文集。新加坡, 1648–1665。[38]维基百科。2023.服务级别协议。(2023)。https://en.wikipedia.org/wiki/Service-level_agreement [在线; 2023 年 10 月 24 日访问]。[39]BigScience Workshop, Teven Le Scao, Angela Fan, Christopher Akiki 等。2023.BLOOM: 176B 参数的开放访问多语言语言模型。arXiv:2211.05100 [cs.CL] [40]Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim 和 Byung-Gon Chun。2022.Orca: 用于{基于 Transformer}生成模型的分布式服务系统。第 16 届 USENIX 操作系统设计与实现研讨会(OSDI 22)。521–538。[41]曾敖涵, 刘晓, 杜正晓, 王子涵, 赖涵宇, 丁明, 杨卓毅, 徐一凡, 郑文迪, 夏晓, 等。2022.Glm-130b: 开放的双语预训练模型。arXiv 预印本 arXiv:2210.02414 (2022)。